

자료구조 및 알고리즘

1. 힙이란?

- 정적 vs 동적 데이터 집합

- 정적 데이터 집합

- 한번 구축되고 나면 변하지 않음

- 동적 데이터 집합

- 데이터가 계속 변함

- Dictionary (Table)

- 삽입, 삭제, 검색을 지원하는 동적 데이터 집합을 지칭

- 배열, 리스트, 검색 트리, 해시 테이블, ...

- 우선 순위 큐

- 삽입, 최우선 원소 삭제, 최우선 원소 검색을 지원하는 동적 데이터 집합

- 배열, 리스트, 검색트리, 힙, ...

- **우선순위 큐와 힙**

- 우선순위가 있는 대상을 관리해야 하는 경우
 - 우선순위를 가진 원소 삽입
 - 우선순위가 가장 큰 원소 빼내기

- 우선순위 큐의 ADT

원소를 삽입한다

최대 원소를 알려주면서 삭제한다

최대 원소를 알려준다

우선순위 큐가 비어 있는지 확인한다

우선순위 큐를 깨끗이 비운다

그림 8-1 ADT 우선순위 큐

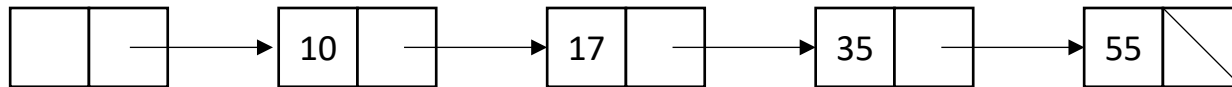
- 최우선 순위는 최대 원소나 최소 원소 중 한 쪽 (대칭)
- 우선순위 큐에서 유일하게 접근 가능한 원소는 최대 원소

- 비효율적인 우선순위 큐

- 배열 리스트

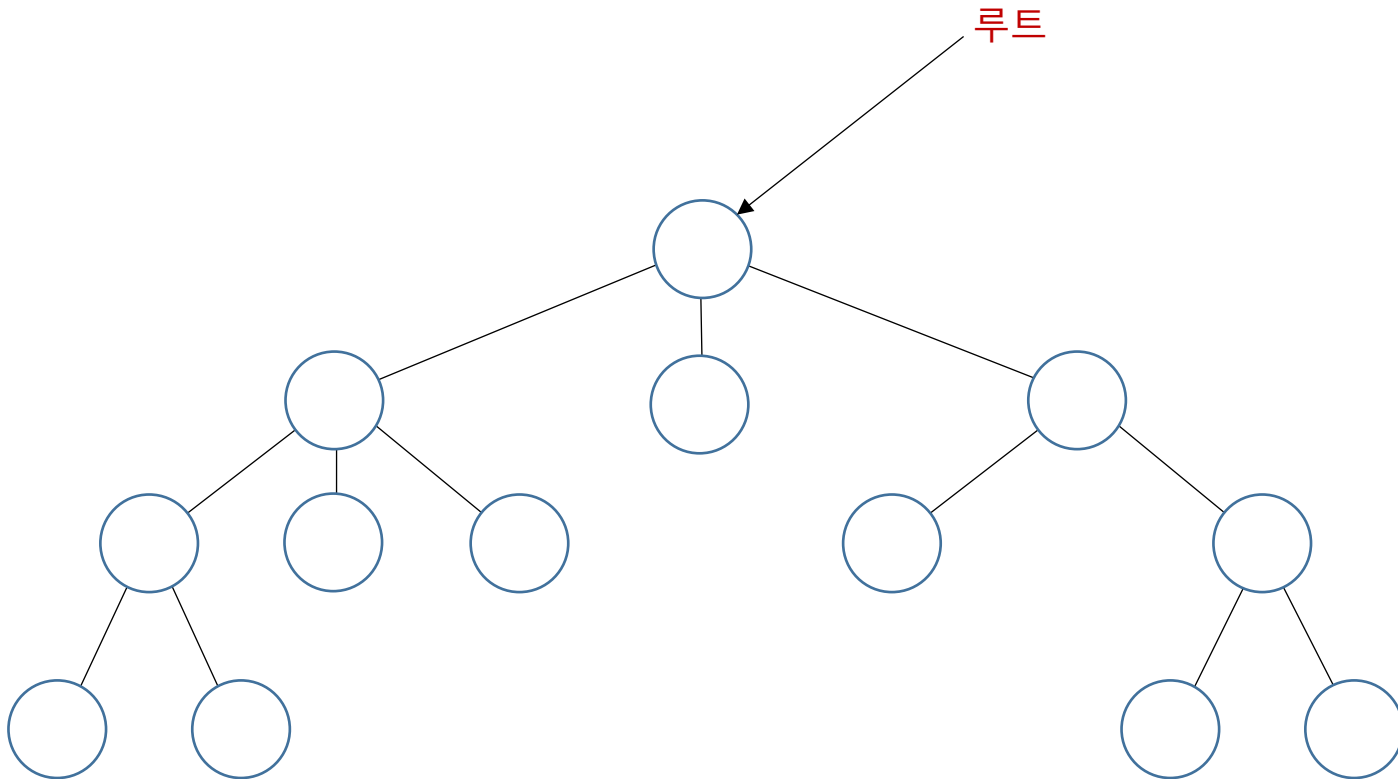
10	35	40	17	95	50	48	33	9			
----	----	----	----	----	----	----	----	---	--	--	--

- 연결 리스트



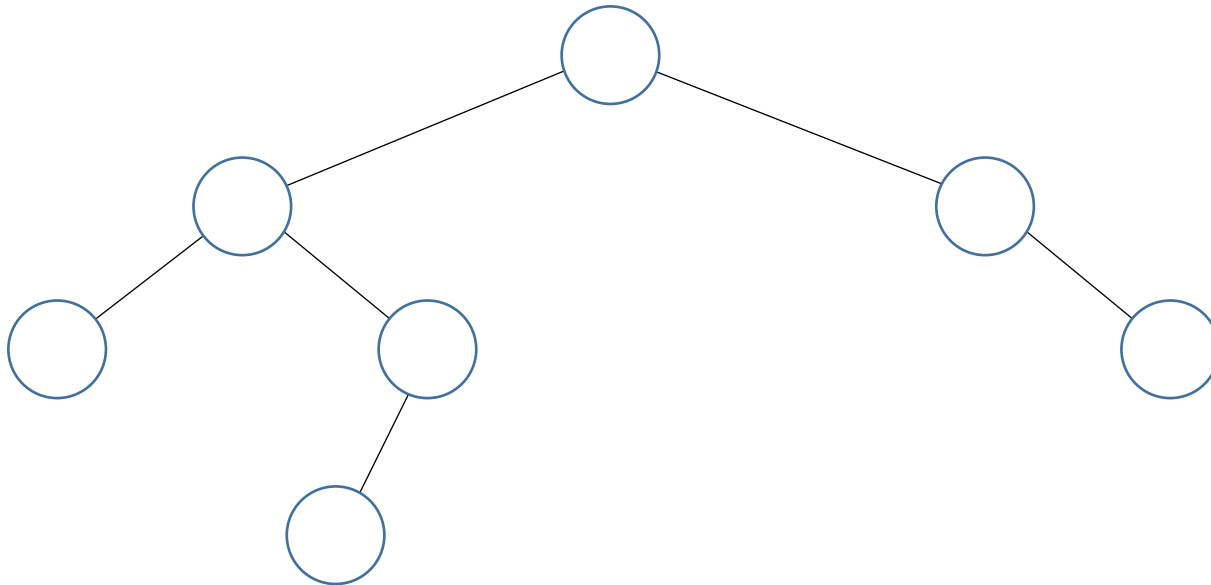
- **힙의 구조**

- 트리 : 부모-자식 관계로 이루어진 노드들



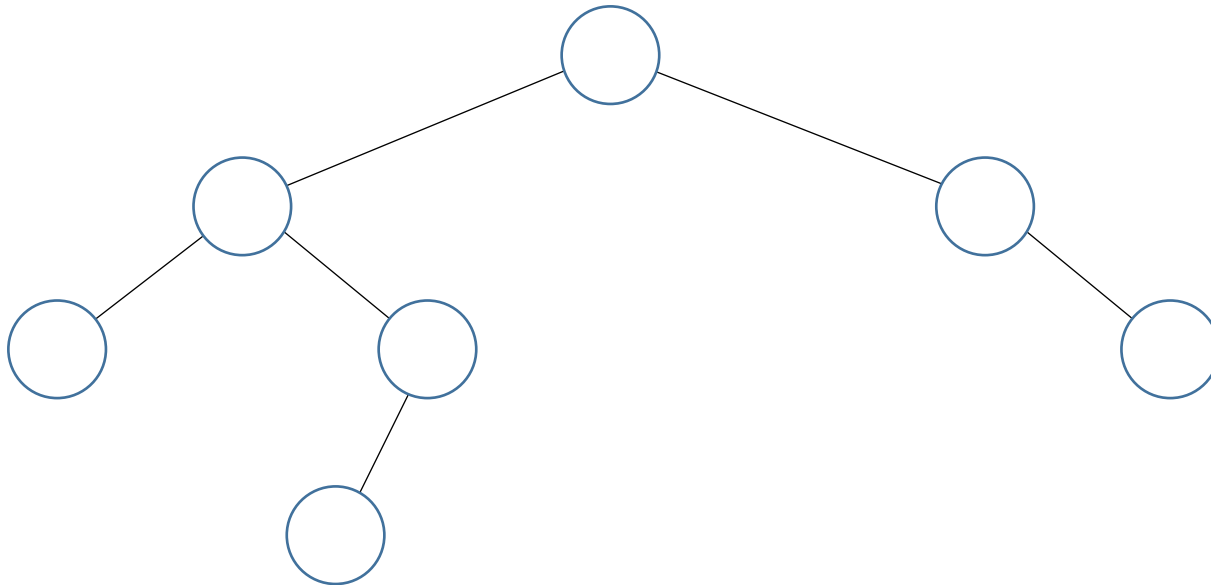
- **힙의 구조**

- 트리 : 부모-자식 관계로 이루어진 노드들
 - 이진 트리: 노드가 2개 이하의 자식을 가짐



- **힙의 구조**

- 트리 : 부모-자식 관계로 이루어진 노드들
 - 이진 트리: 노드가 2개 이하의 자식을 가짐
 - 포화 이진 트리 : 모든 노드가 정확히 자식 노드를 2개씩 가지면서 꽉 채워진 트리

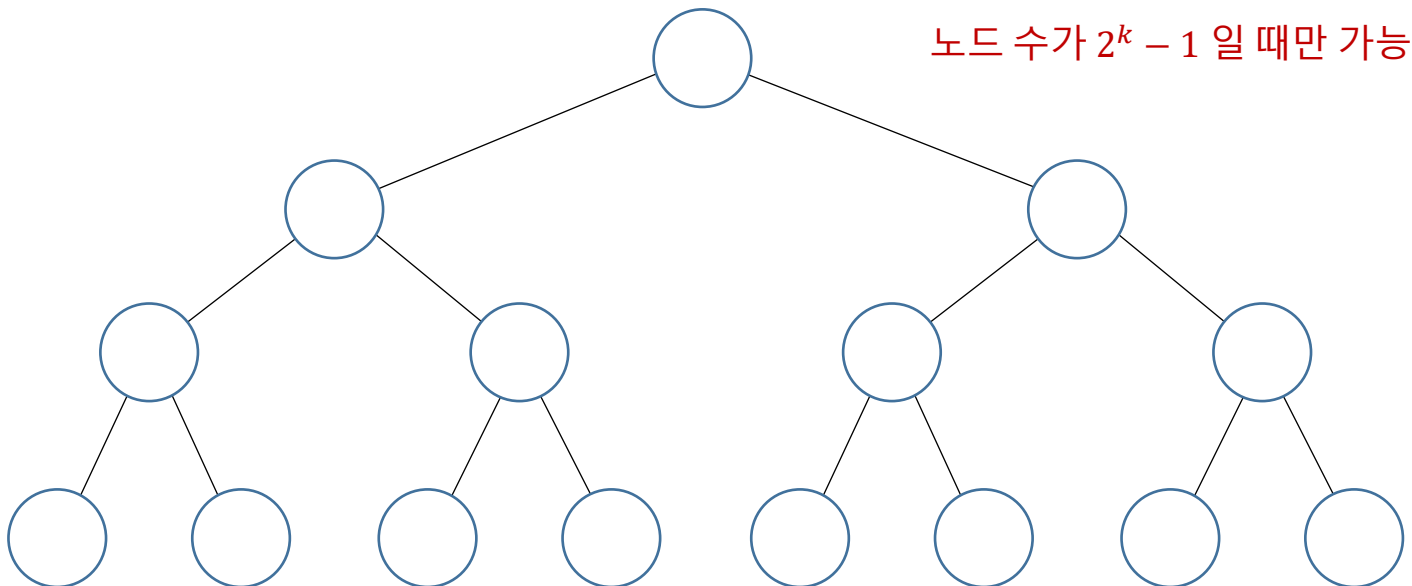


- **힙의 구조**

- 트리 : 부모-자식 관계로 이루어진 노드들

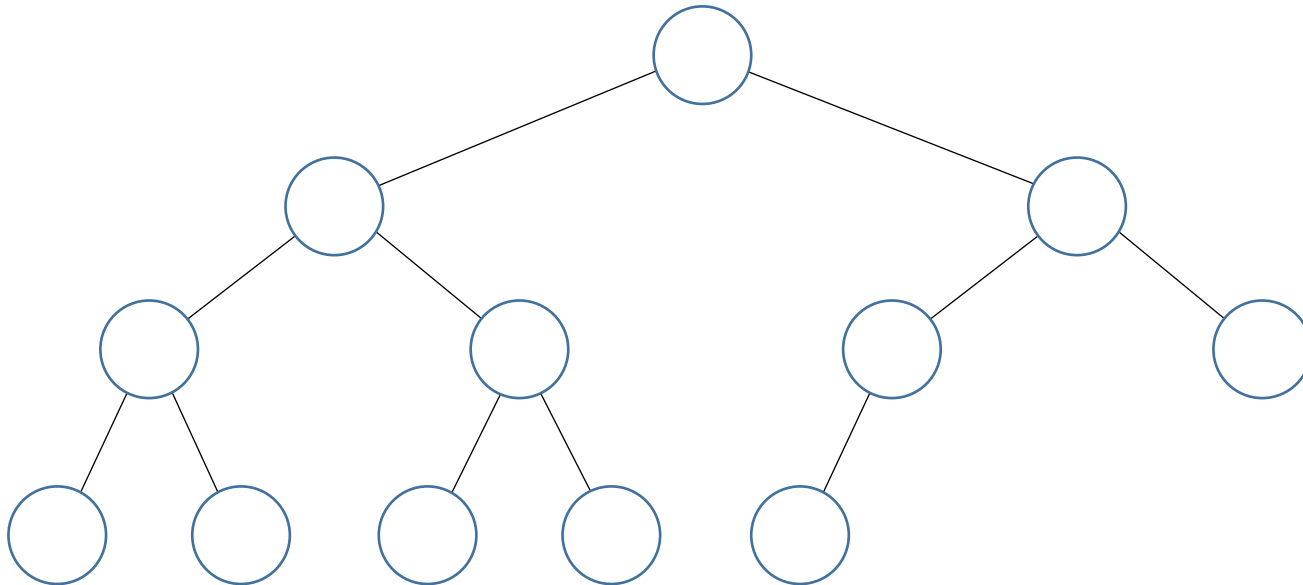
- 이진 트리: 노드가 2개 이하의 자식을 가짐

- **포화 이진 트리** : 모든 노드가 정확히 자식 노드를 2개씩 가지면서 꽉 채워진 트리



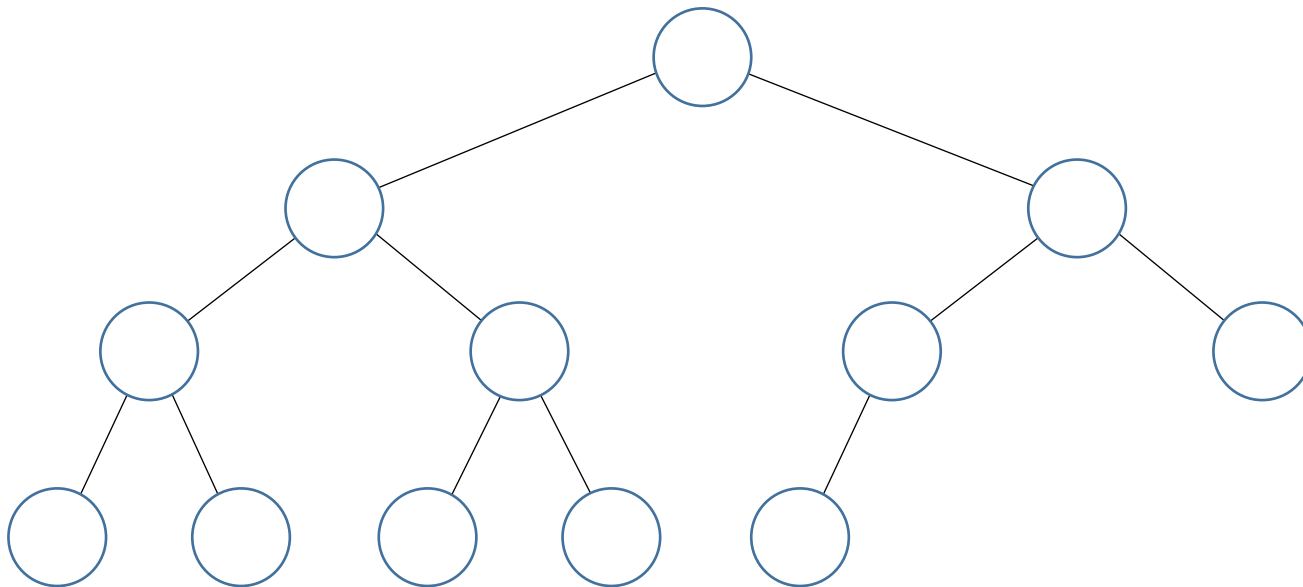
■ 힙의 구조

- 트리 : 부모-자식 관계로 이루어진 노드들
 - 이진 트리: 노드가 2개 이하의 자식을 가짐
 - 포화 이진 트리 : 모든 노드가 정확히 자식 노드를 2개씩 가지면서 꽉 채워진 트리
 - **완전 이진 트리** : 최대한 포화 이진 트리에 가깝게 만든 트리
 - 루트로부터 시작해서 가능한 지점까지 모든 노드가 정확히 2개의 자식 노드를 가짐
 - 맨 마지막 레벨은 왼쪽부터 채워 나감



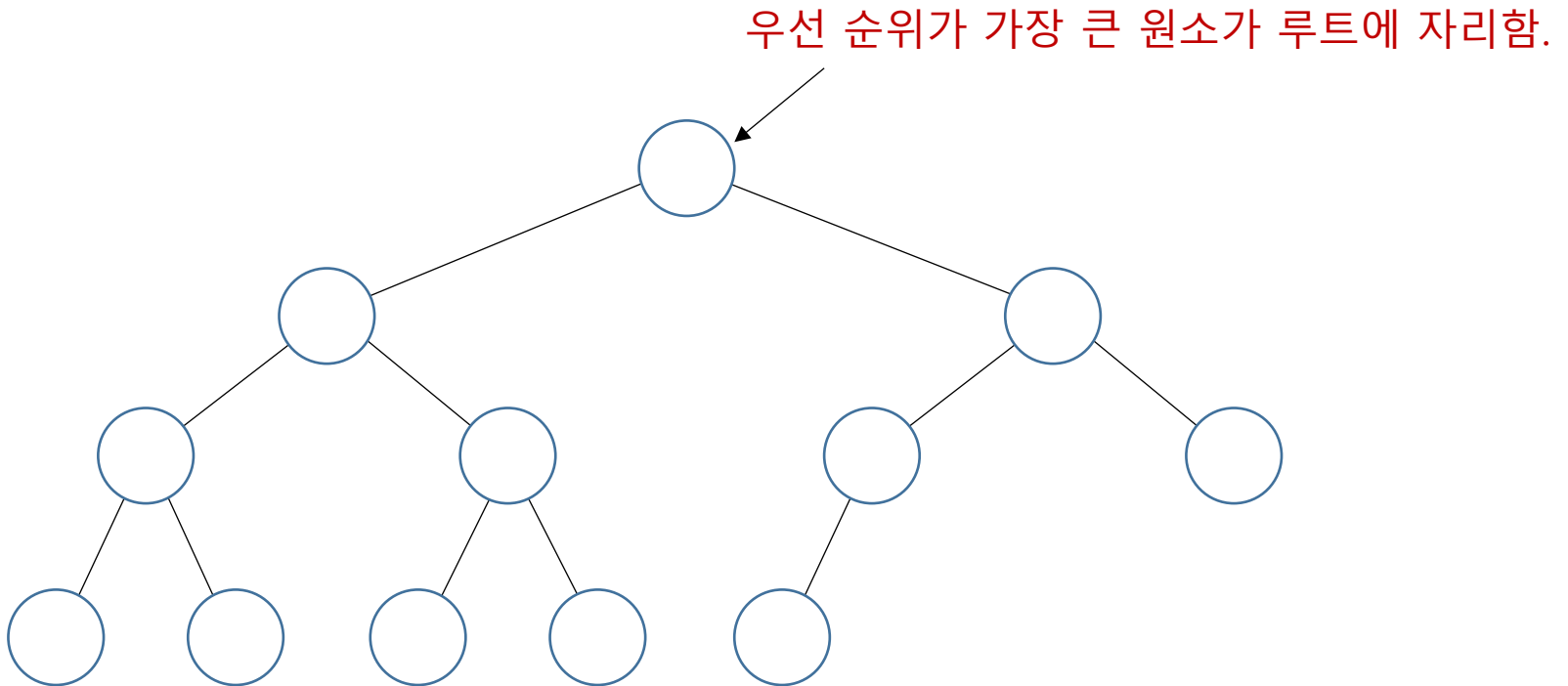
- **힙의 조건**

- 완전 이진 트리
- 힙 특성 : 모든 노드는 값을 갖고, 자식 노드 값보다 크거나 같다. (Max heap)



▪ 힙의 조건

- 완전 이진 트리
- 힙 특성 : 모든 노드는 값을 갖고, 자식 노드 값보다 크거나 같다. (Max heap)

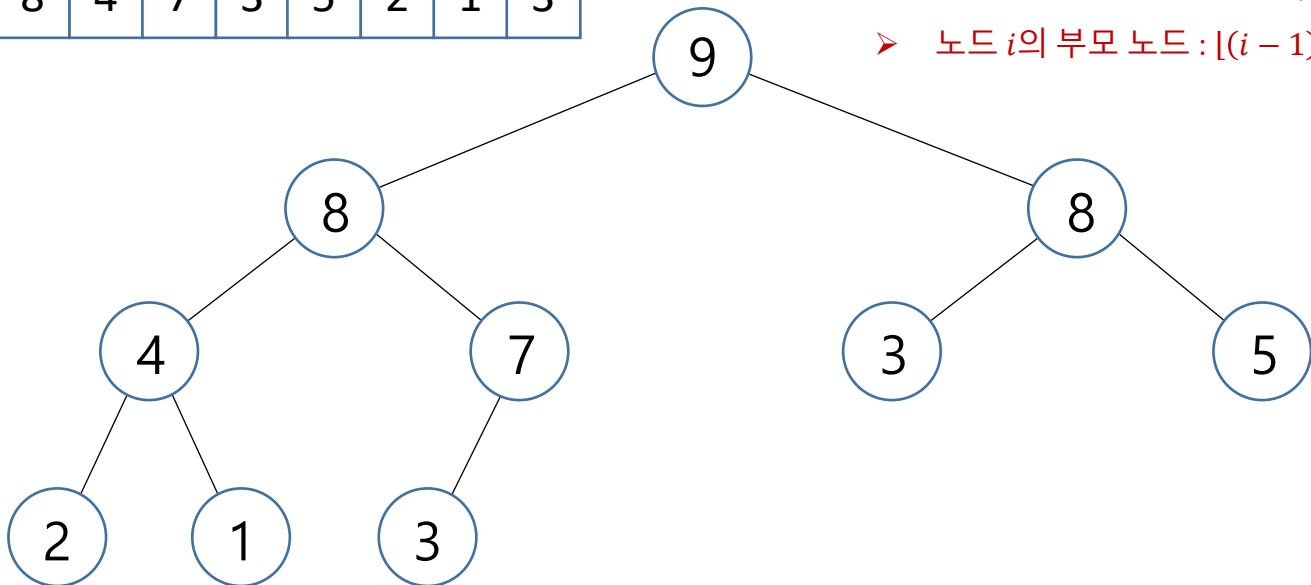


▪ 힙의 조건

- 완전 이진 트리
- 힙 특성 : 모든 노드는 값을 갖고, 자식 노드 값보다 크거나 같다. (Max heap)

A : 리스트 → **항상 완전 이진 트리로 간주할 수 있음!**

9	8	8	4	7	3	5	2	1	3
---	---	---	---	---	---	---	---	---	---



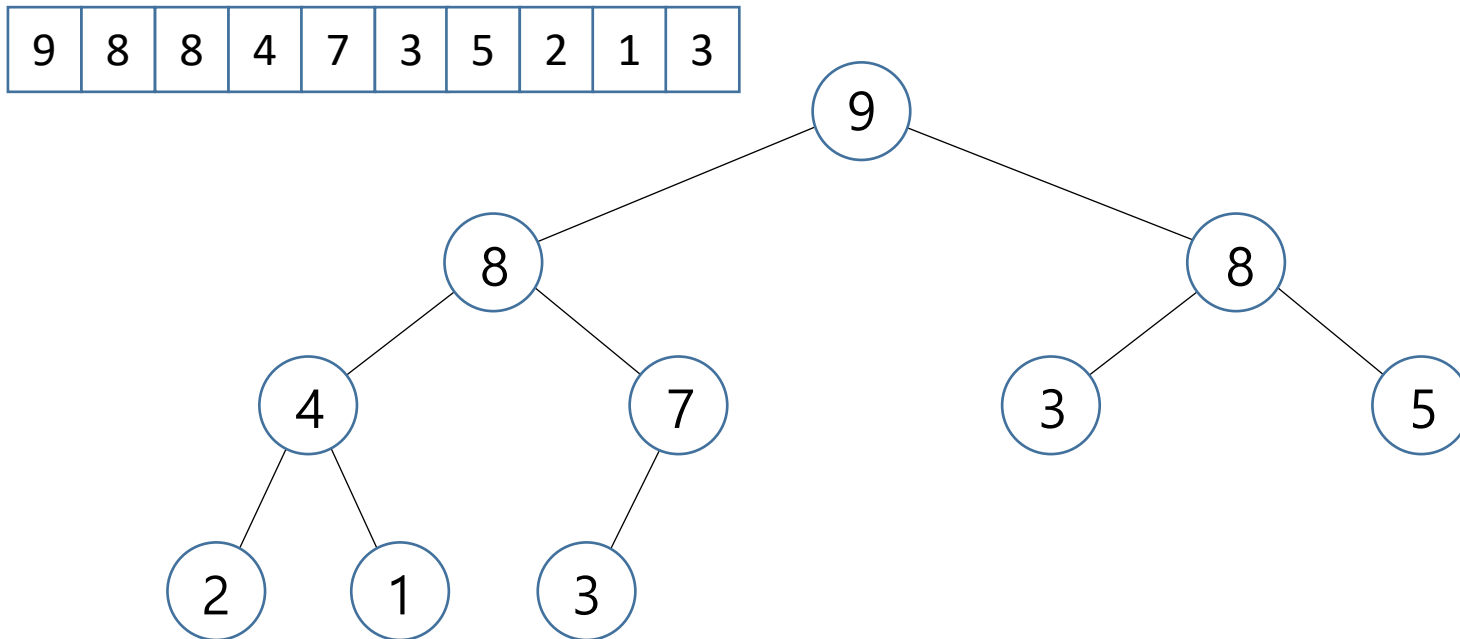
➤ 노드 i 의 자식 노드 : $2i + 1, 2i + 2$

➤ 노드 i 의 부모 노드 : $\lfloor (i - 1) / 2 \rfloor$

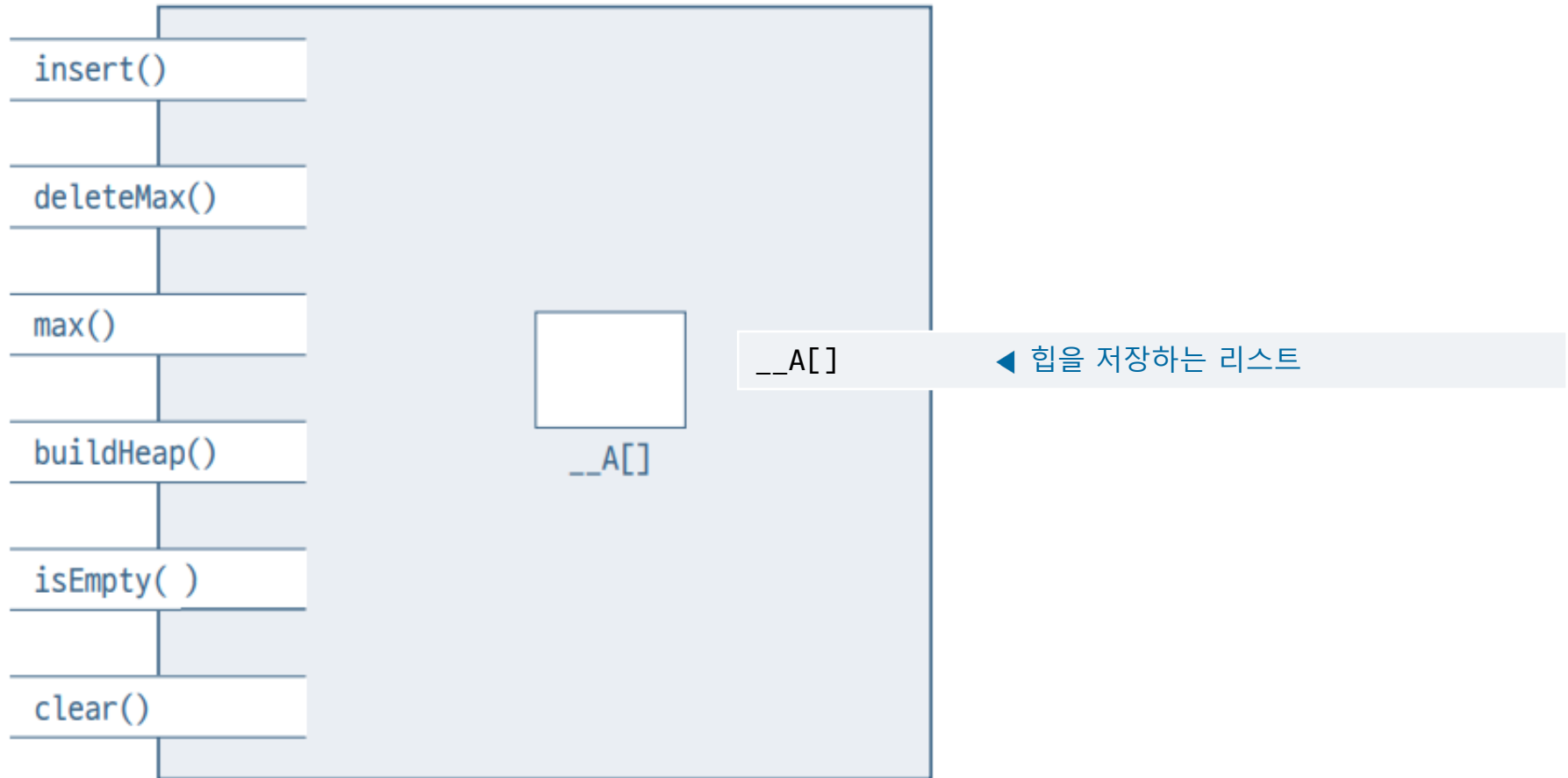
▪ 힙의 조건

- ~~완전 이진 트라~~(리스트를 사용)
- 힙 특성 : 모든 노드는 값을 갖고, 자식 노드 값보다 크거나 같다. (Max heap)

→ 리스트가 힙 특성을 만족하도록 수선!



▪ 힙의 객체 구조



▪ 힙의 객체 구조

insert()	insert(x) ◀ 힙에 원소 x를 삽입한다.
deleteMax()	deleteMax() ◀ 힙의 최대 원소를 알려주면서 삭제한다.
max()	max() ◀ 힙의 최대 원소를 알려준다.
buildHeap()	buildHeap() ◀ 리스트 A[]를 힙으로 만든다.
isEmpty()	isEmpty() ◀ 힙이 비었는지 알려준다.
clear()	clear() ◀ 힙을 깨끗이 청소한다.

2. 힙 작업 알고리즘과 구현

- 힙의 작업과 구현

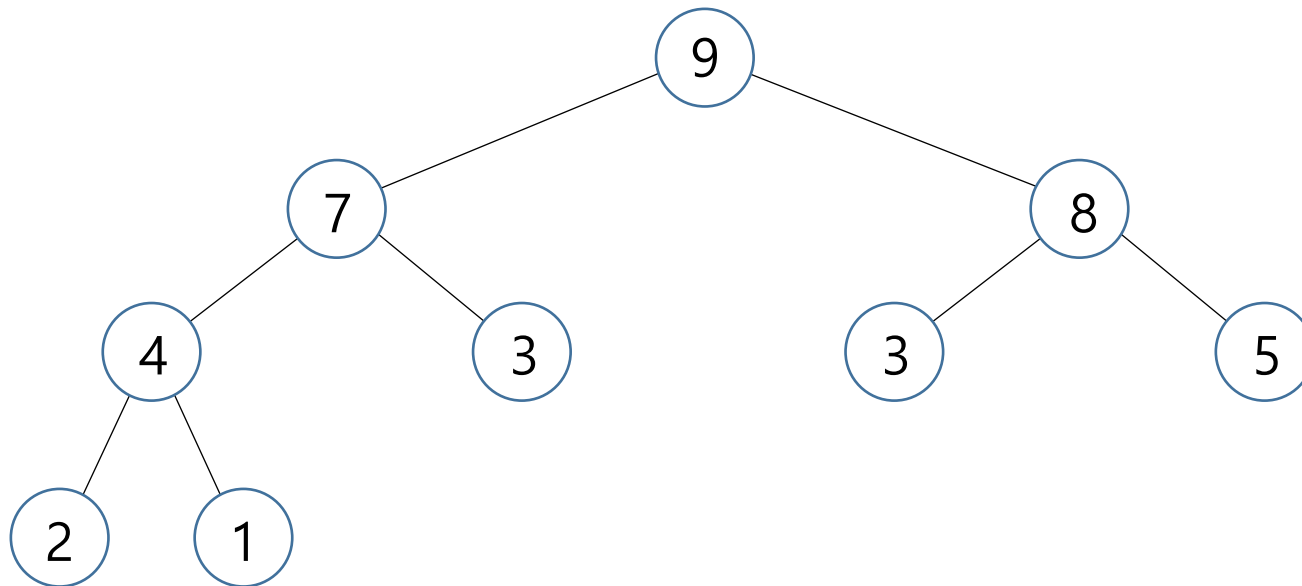
- heap.py

```
class Heap:
    def __init__(self, list):
        if list == None:
            self.__A = []
        else:
            self.__A = list

    def insert(self, x): ...
    def deleteMax(self): ...
    def max(self): ...
    def size(self): ...
    def isEmpty(self): ...
    def clear(self): ...
```

- 힙의 작업과 구현

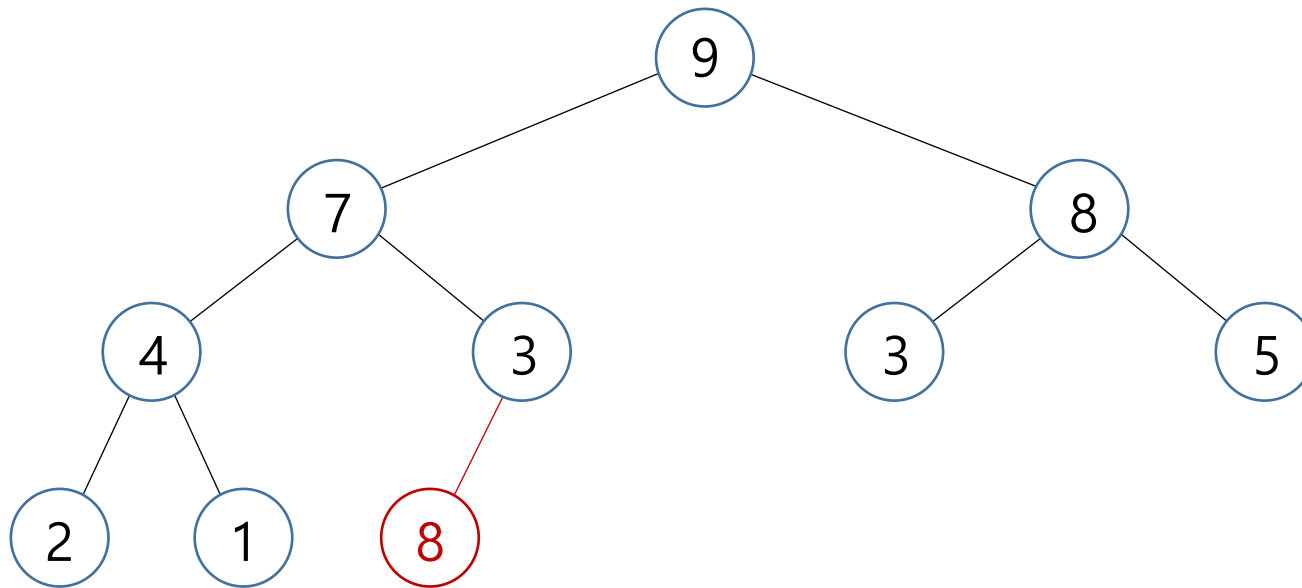
- 원소 삽입



9	7	8	4	3	3	5	2	1
---	---	---	---	---	---	---	---	---

- 힙의 작업과 구현

- 원소 삽입

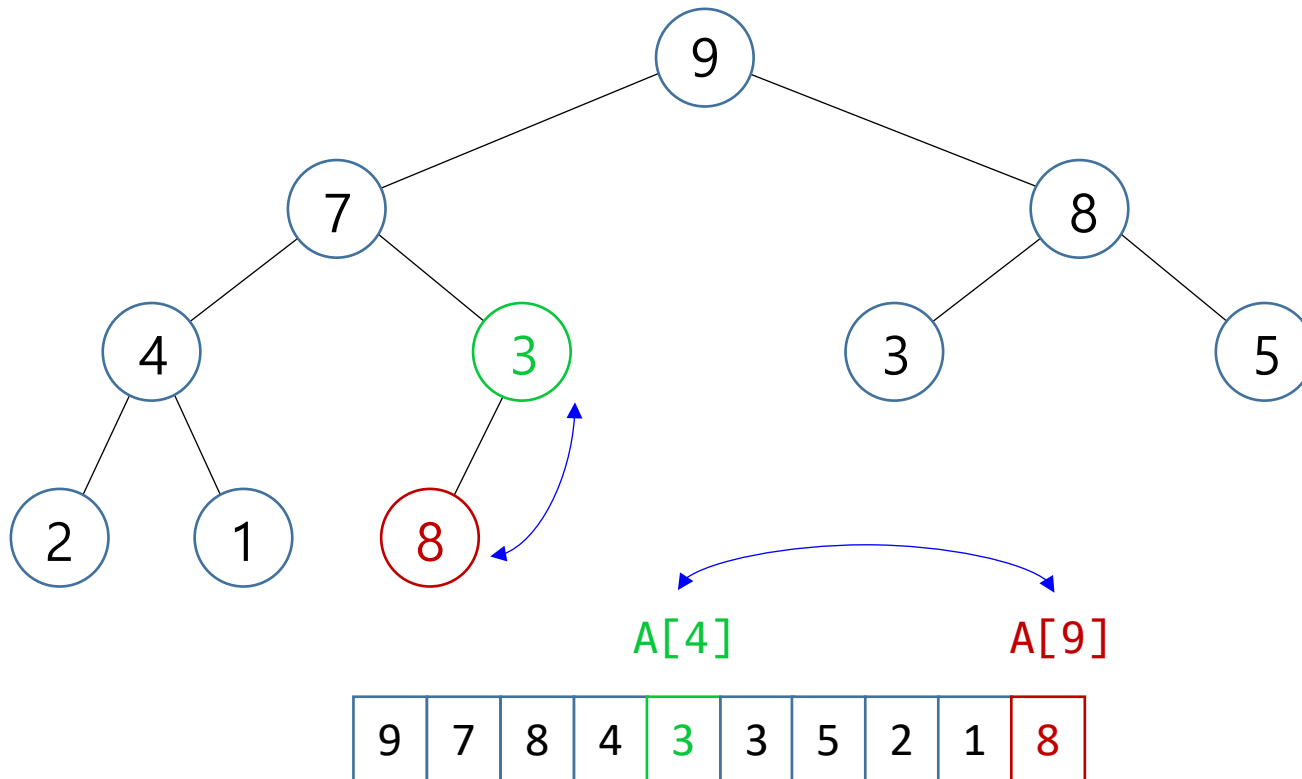


$A[9] = 8$

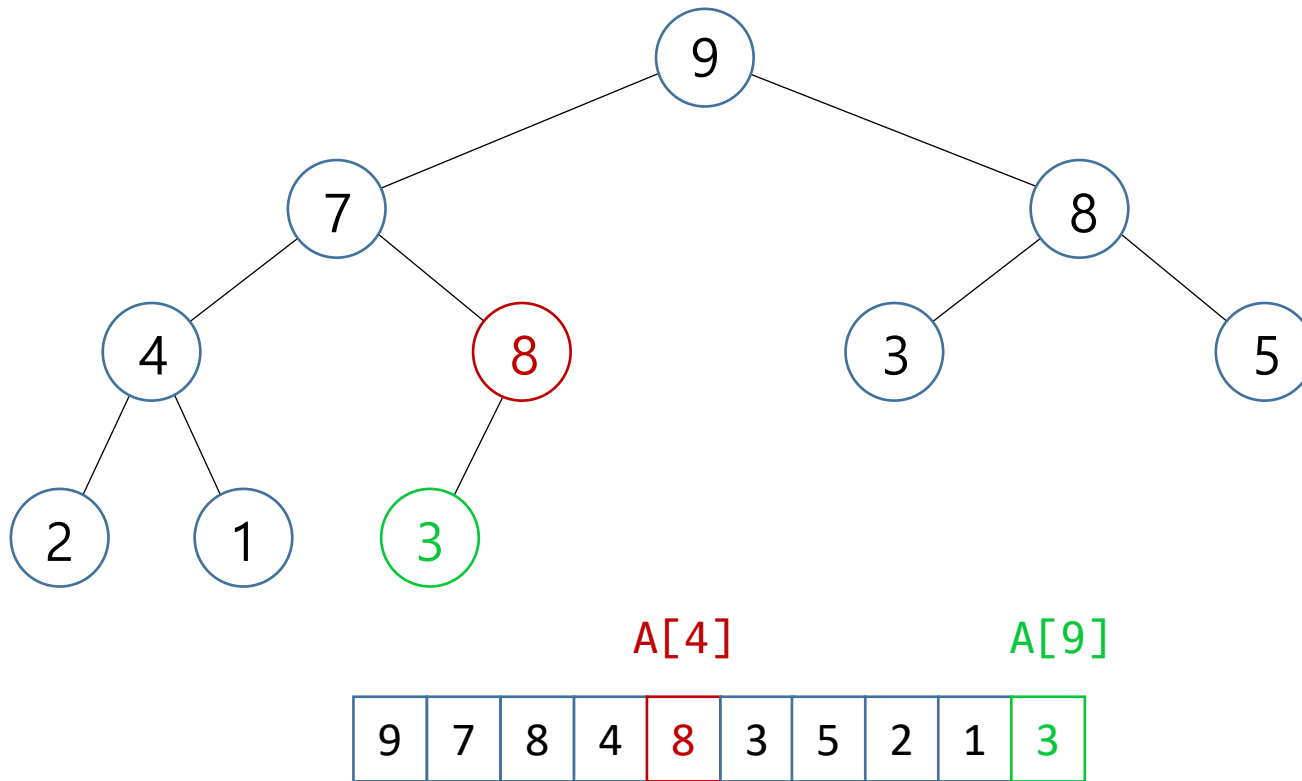
9	7	8	4	3	3	5	2	1	8
---	---	---	---	---	---	---	---	---	---

- 힙의 작업과 구현

- 원소 삽입

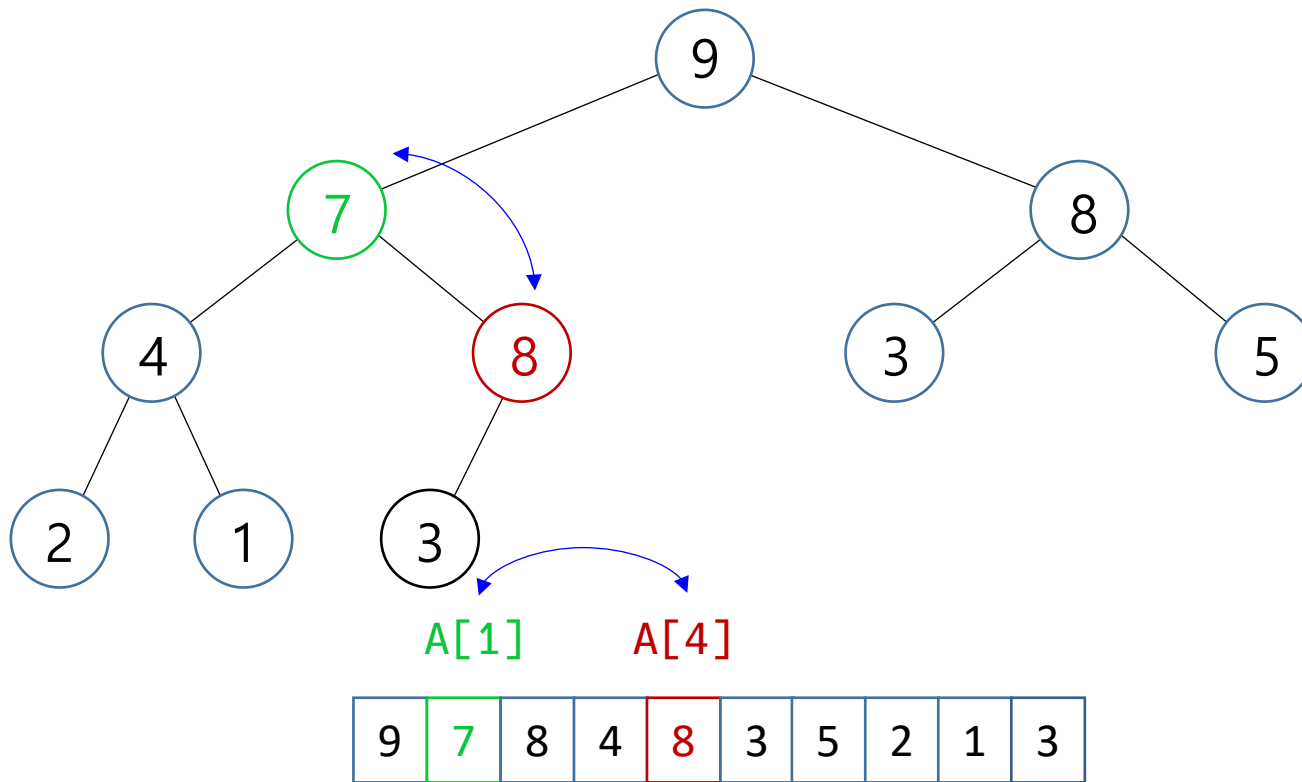


- 힙의 작업과 구현
 - 원소 삽입



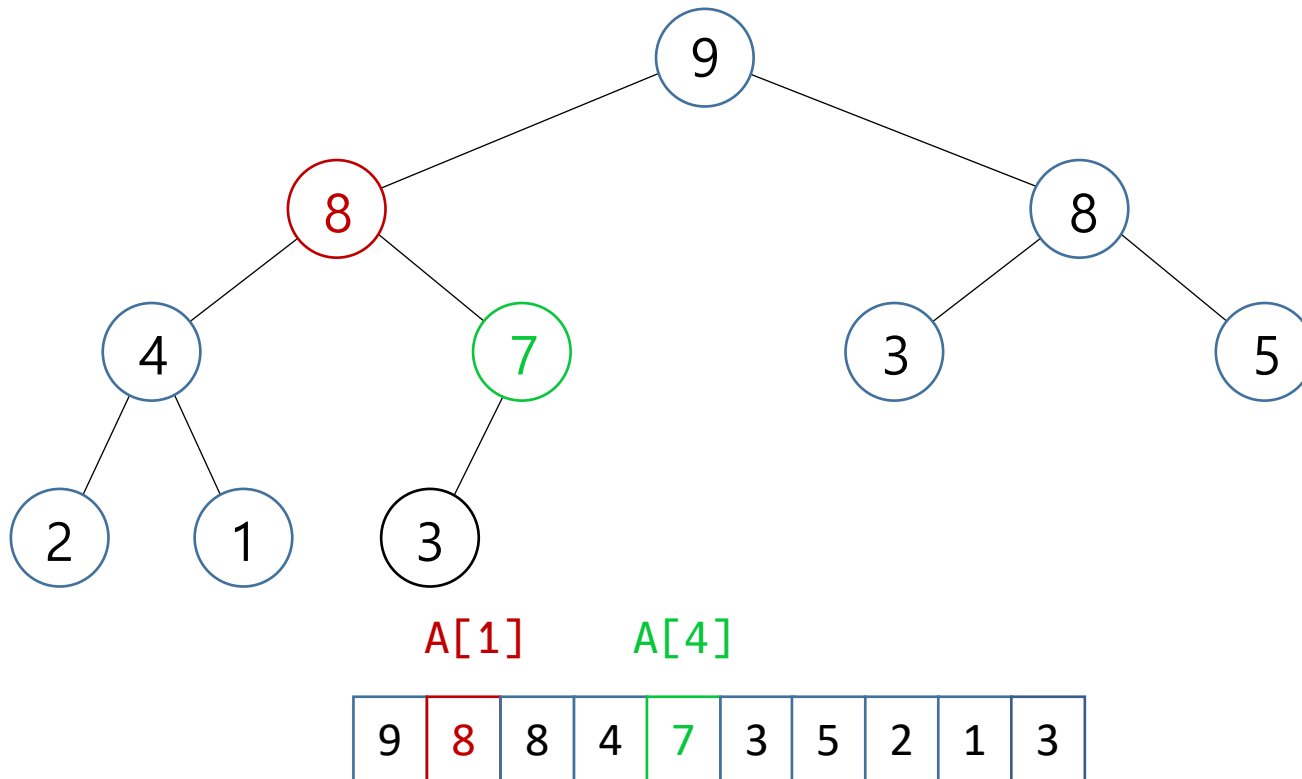
- 힙의 작업과 구현

- 원소 삽입



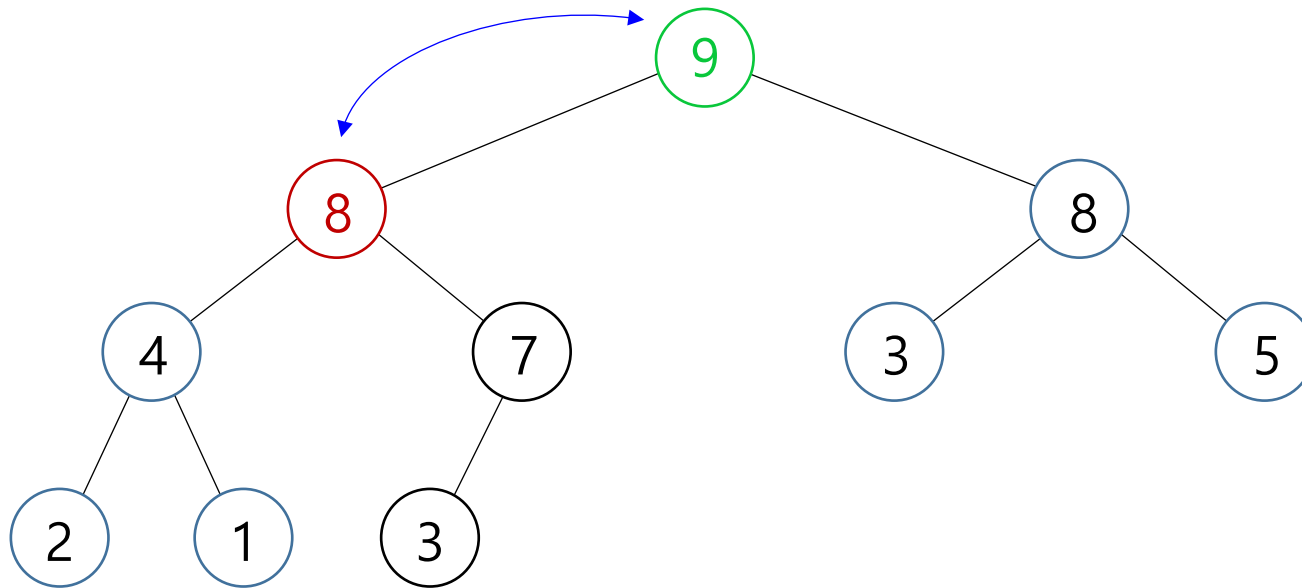
- 힙의 작업과 구현

- 원소 삽입



- 힙의 작업과 구현

- 원소 삽입

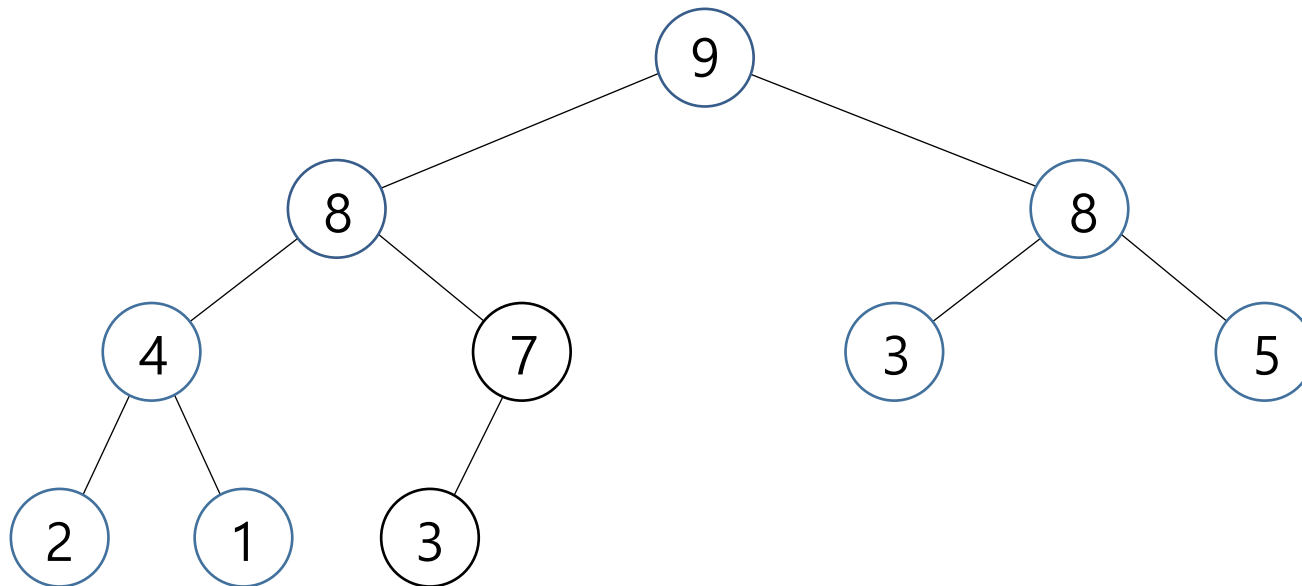


A[0] A[1]

9	8	8	4	7	3	5	2	1	3
---	---	---	---	---	---	---	---	---	---

- 힙의 작업과 구현

- 원소 삽입



9	8	8	4	7	3	5	2	1	3
---	---	---	---	---	---	---	---	---	---

- 힙의 작업과 구현

- 원소 삽입

알고리즘 8-1 힙에 원소 삽입하기

◀ 힙 $A[0 \dots n-1]$ 에 원소를 삽입한다(추가한다)

insert(x): ◀ x : 삽입할 원소

$i \leftarrow n$

$A[i] \leftarrow x$

$parent \leftarrow \lfloor (i-1)/2 \rfloor$

 while ($i > 0$ and $A[i] > A[parent]$)

$A[i] \leftrightarrow A[parent]$ ◀ 맞바꾸기

$i \leftarrow parent$

$parent \leftarrow \lfloor (i-1)/2 \rfloor$

 n++ ◀ 힙 크기 1 증가

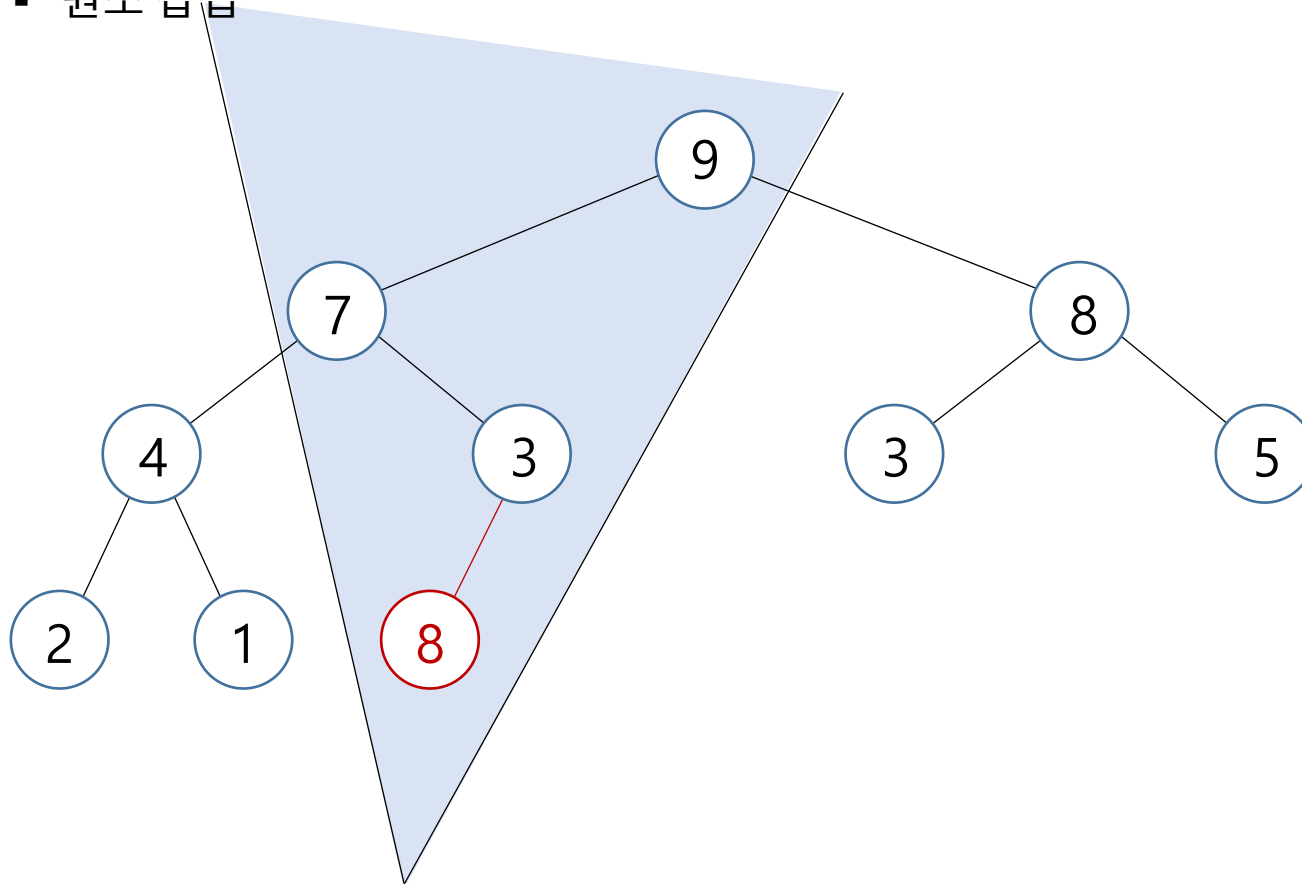
- **힙의 작업과 구현**

- 원소 삽입

```
def insert(self, x):  
    i = len(self.__A)  
    self.__A[i] = x  
    parent = (i-2) // 2  
    while i>0 and A[i] > A[parent]  
        self.__A[i], self.__A[parent] = self.__A[parent], self.__A[i]  
        i = parent  
        parent = (i-2) // 2
```

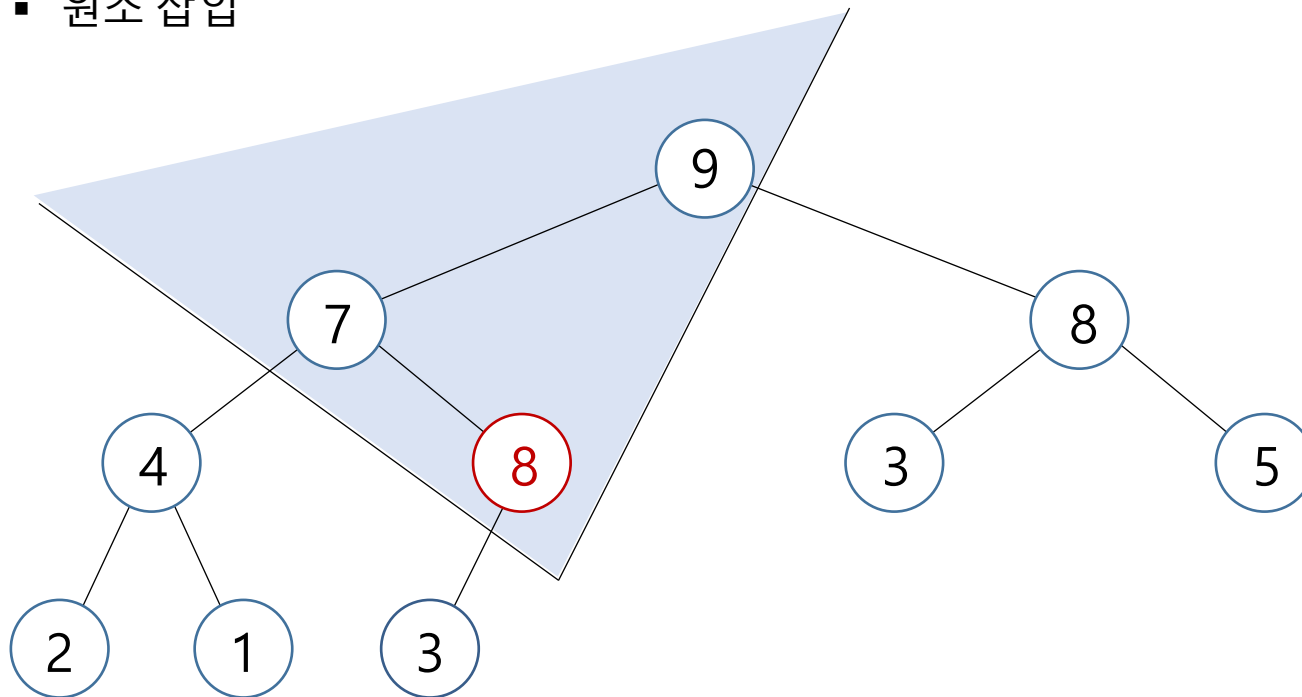
▪ 힙의 작업과 구현

▪ 원소 삽입



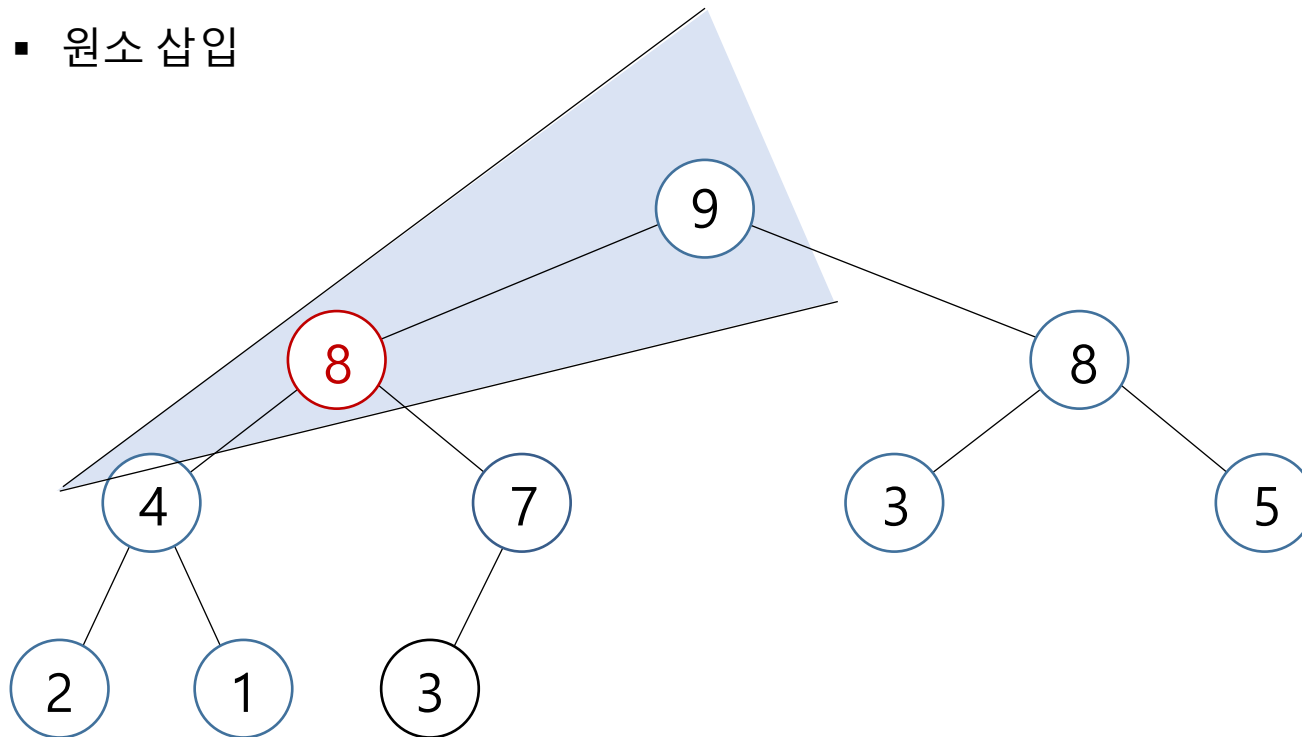
- 힙의 작업과 구현

- 원소 삽입



- 힙의 작업과 구현

- 원소 삽입



- 힙의 작업과 구현

- 원소 삽입

◀ 힙 $A[0 \dots n-1]$ 에 원소를 삽입한다(추가한다)

`insert(x):`

◀ x : 삽입할 원소

$A[n] \leftarrow x$

`percolateUp(n)`

$n++$

`percolateUp(i):`

$parent \leftarrow (i-1)/2$

`if ( $i > 0$  and  $A[i] > A[parent]$ )`

$A[i] \leftrightarrow A[parent]$

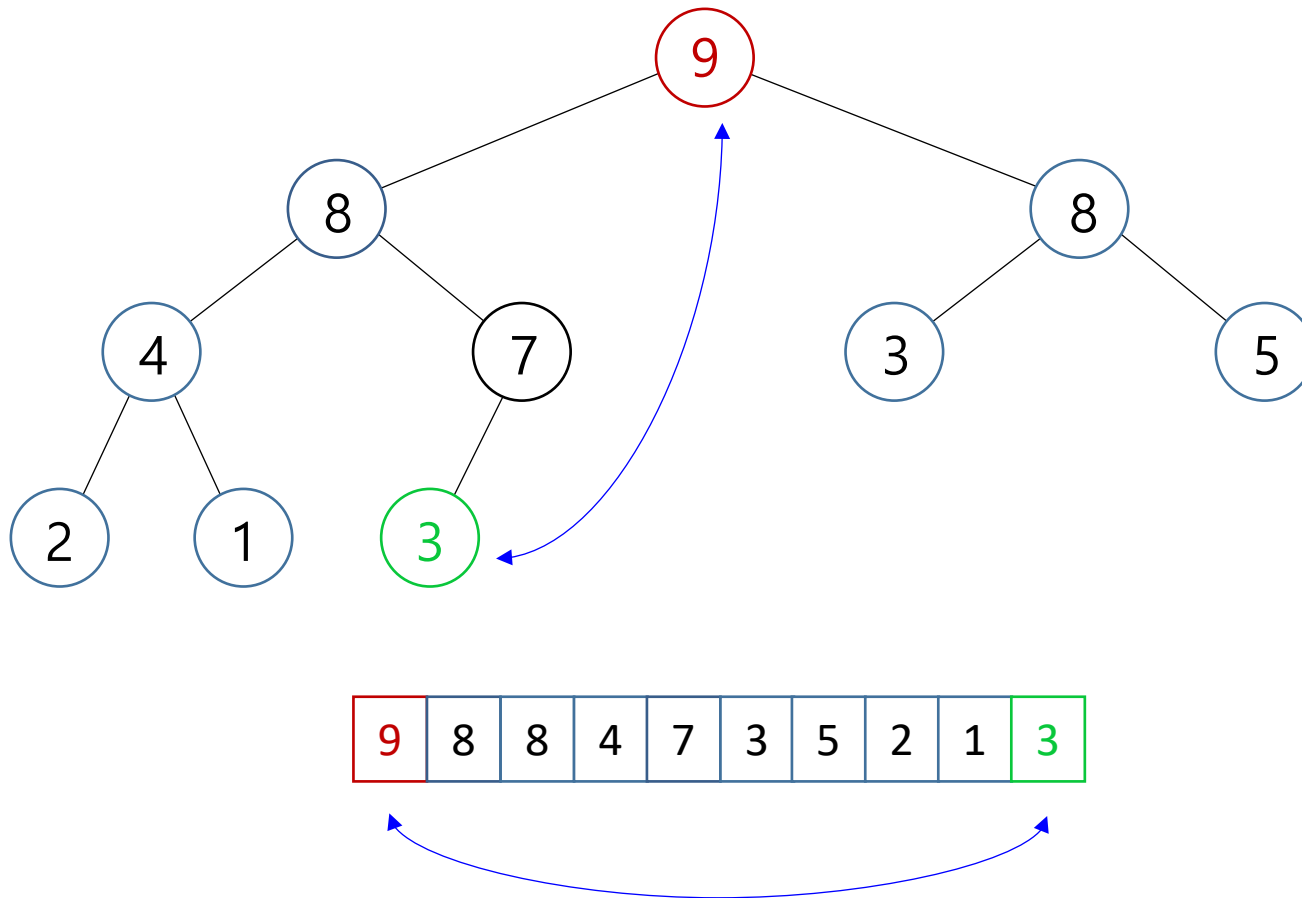
`percolateUp(parent)`

- **힙의 작업과 구현**

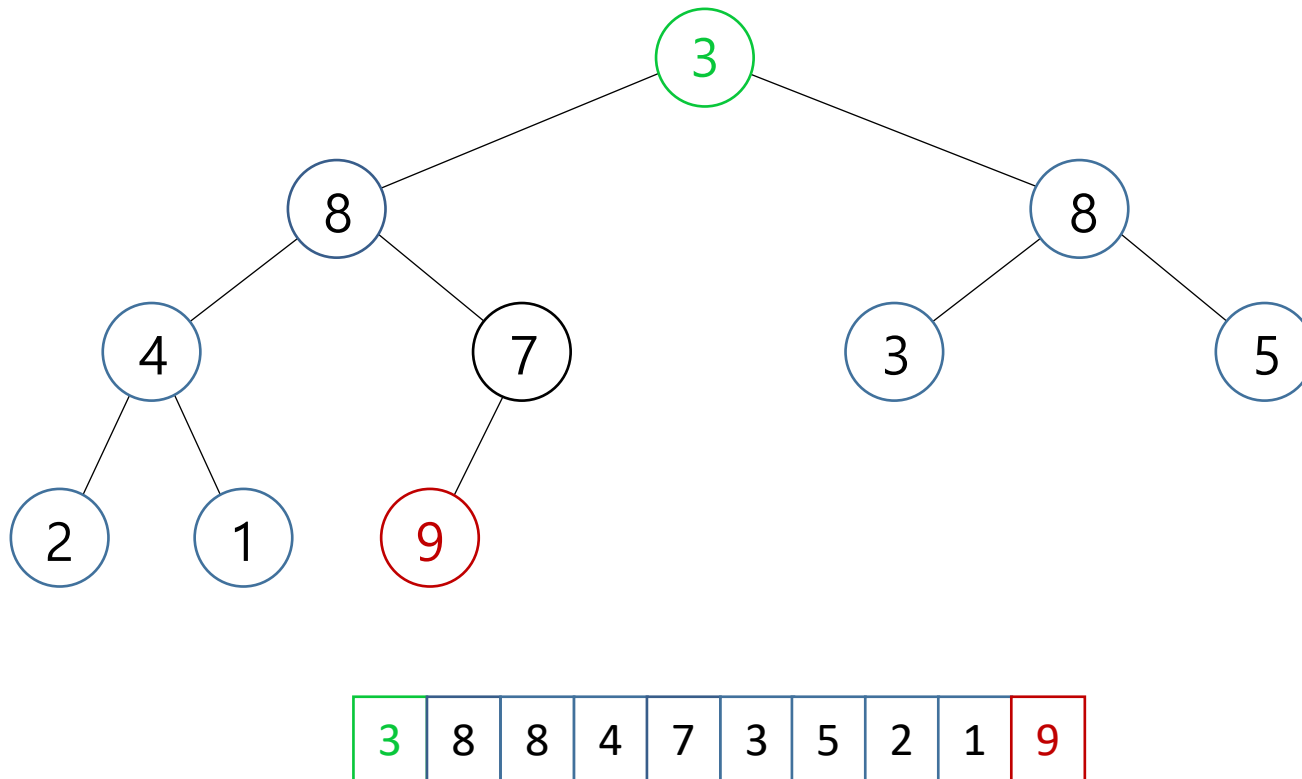
- 원소 삽입

```
def insert(self, x):  
    self.__A.append(x)  
    self.percolateUp(len(self.__A)-1)  
  
def percolateUp(self, i:int):  
    parent = (i-1) // 2  
    if i>0 and self.__A[i] > self.__A[parent]:  
        self.__A[i], self.__A[parent] = self.__A[parent], self.__A[i]  
        self.percolateUp(parent)
```


- 힙의 작업과 구현
 - 원소 삭제

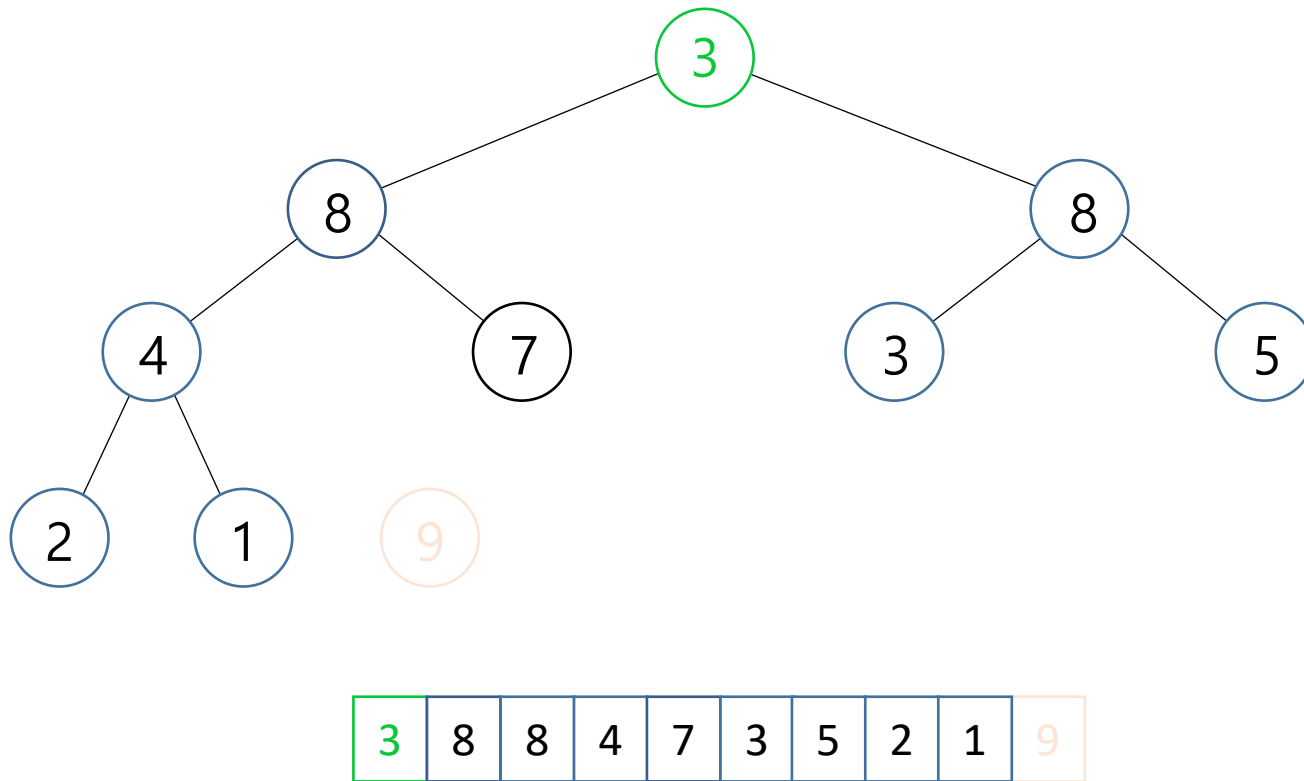


- 힙의 작업과 구현
 - 원소 삭제

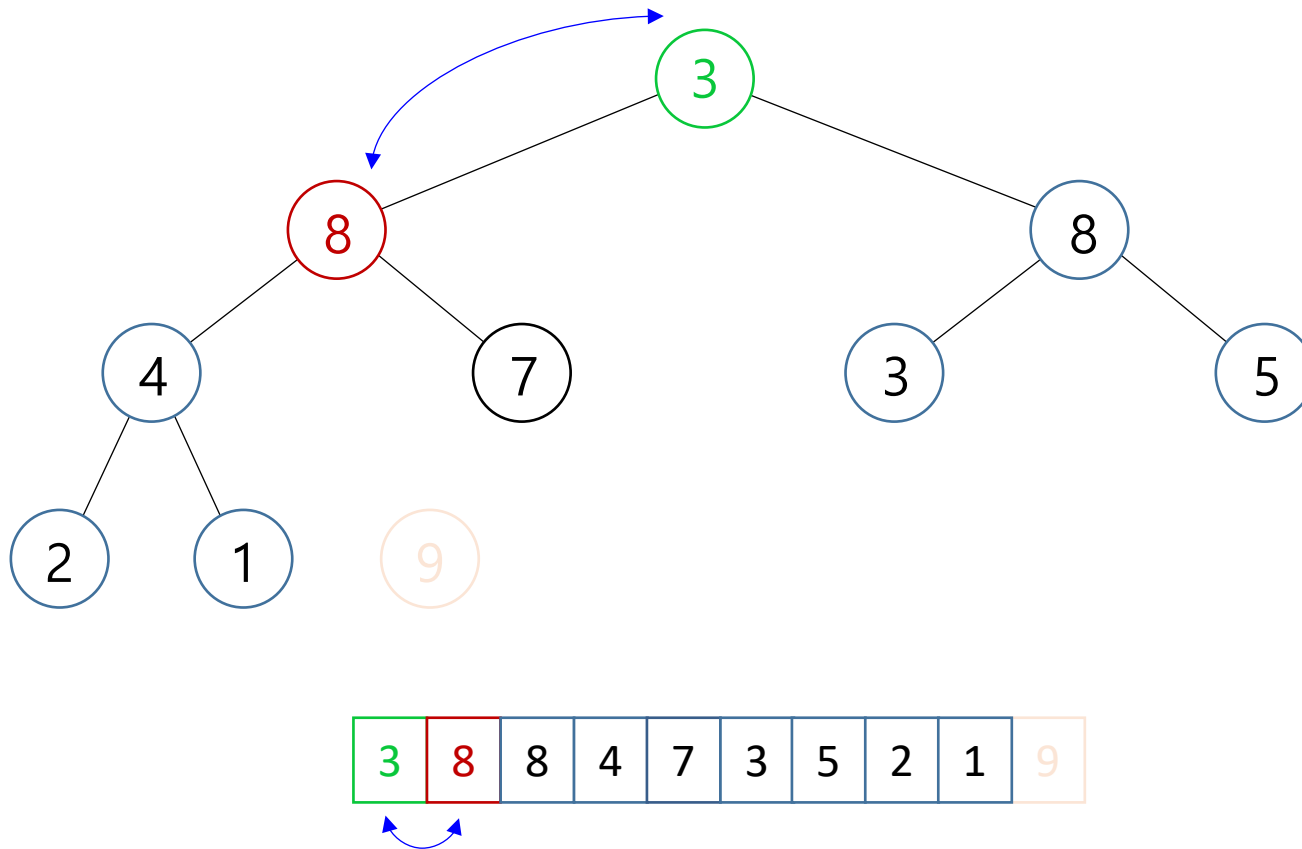


- 힙의 작업과 구현

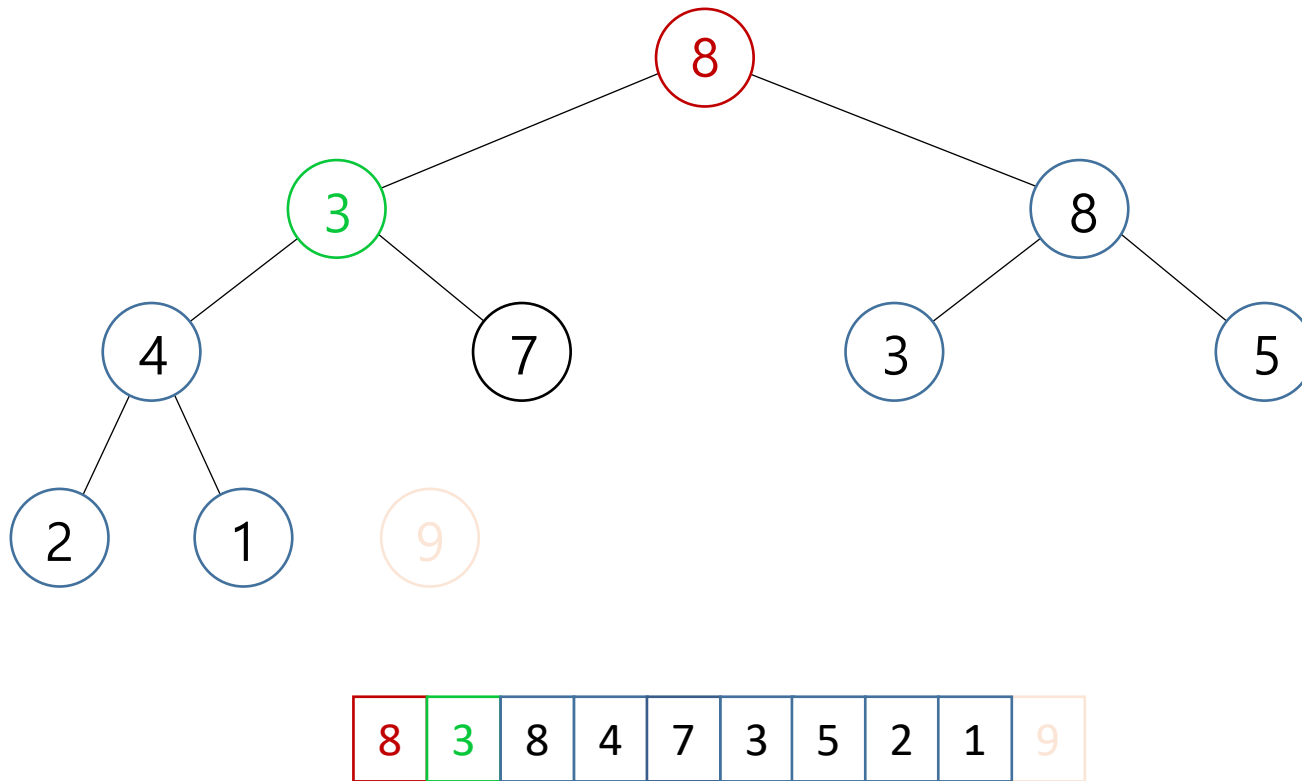
- 원소 삭제



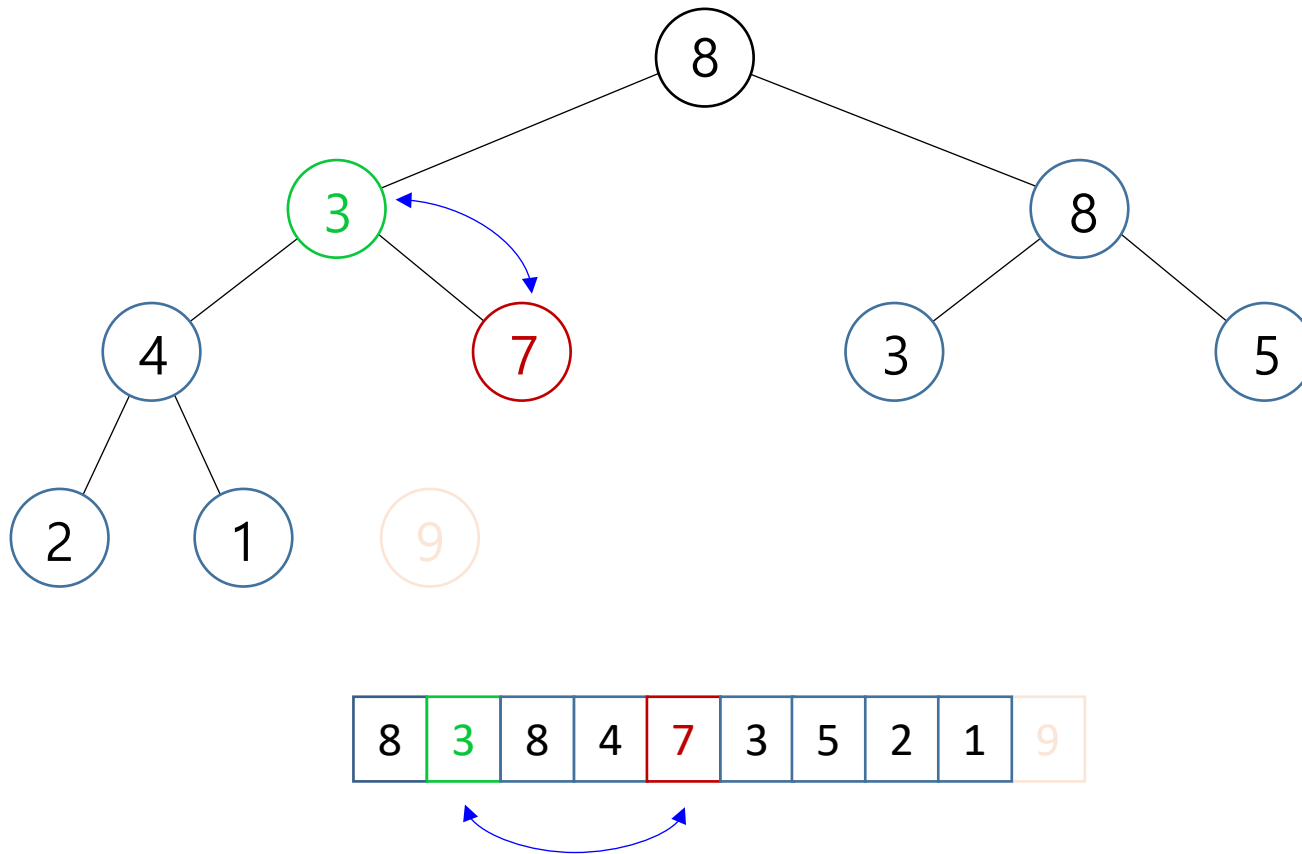
- 힙의 작업과 구현
 - 원소 삭제



- 힙의 작업과 구현
 - 원소 삭제

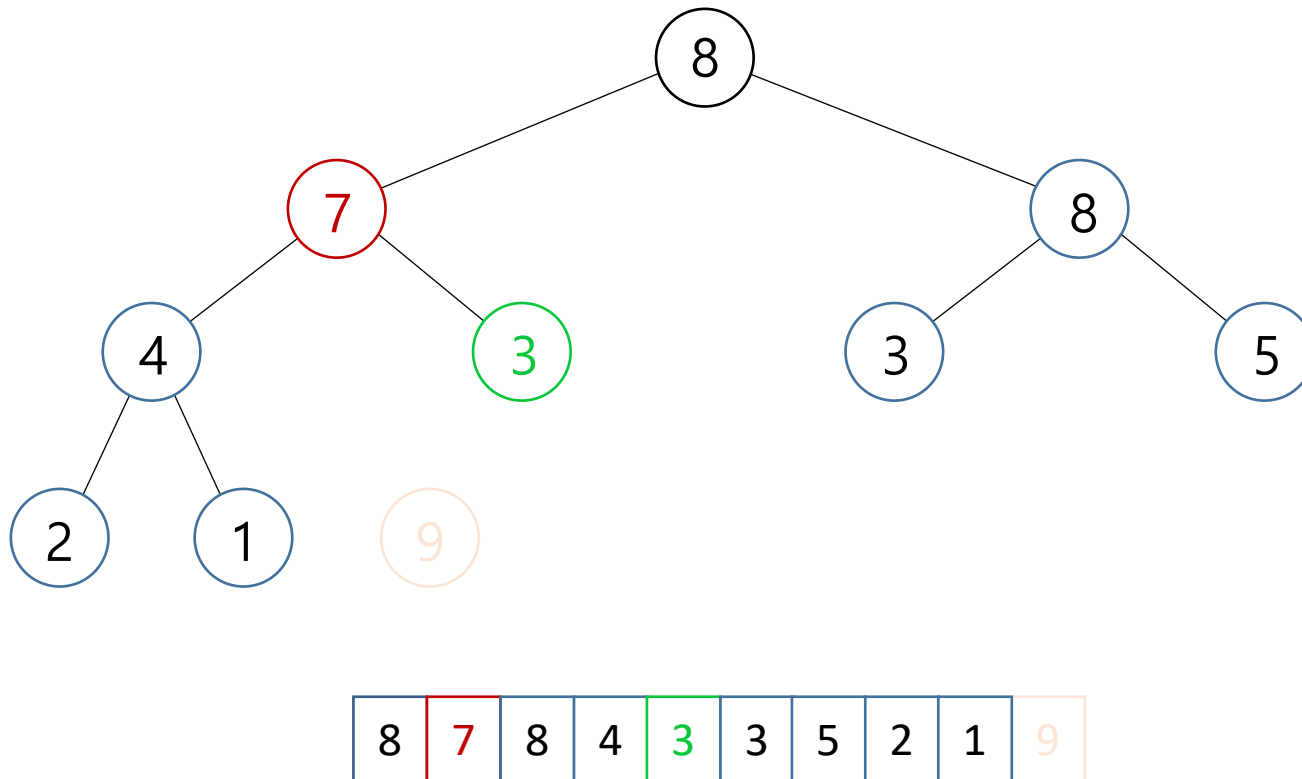


- 힙의 작업과 구현
 - 원소 삭제

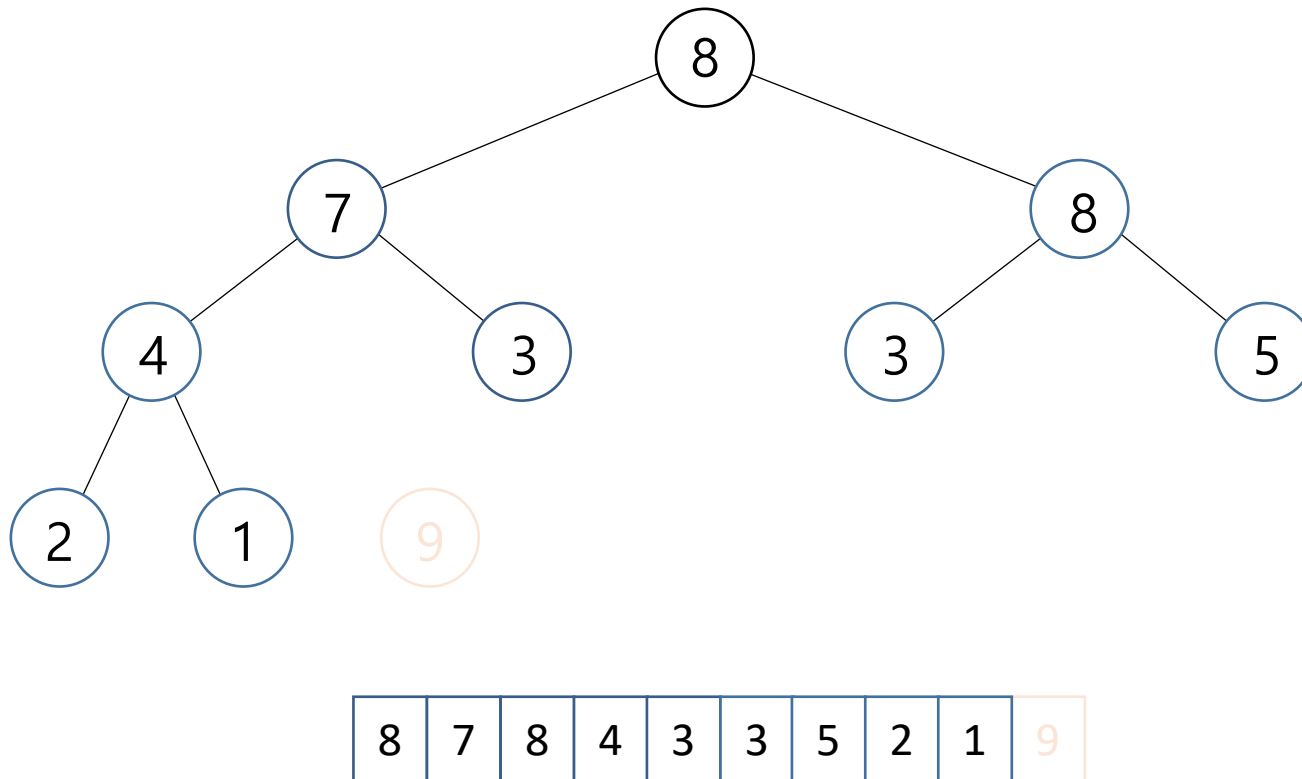


- 힙의 작업과 구현

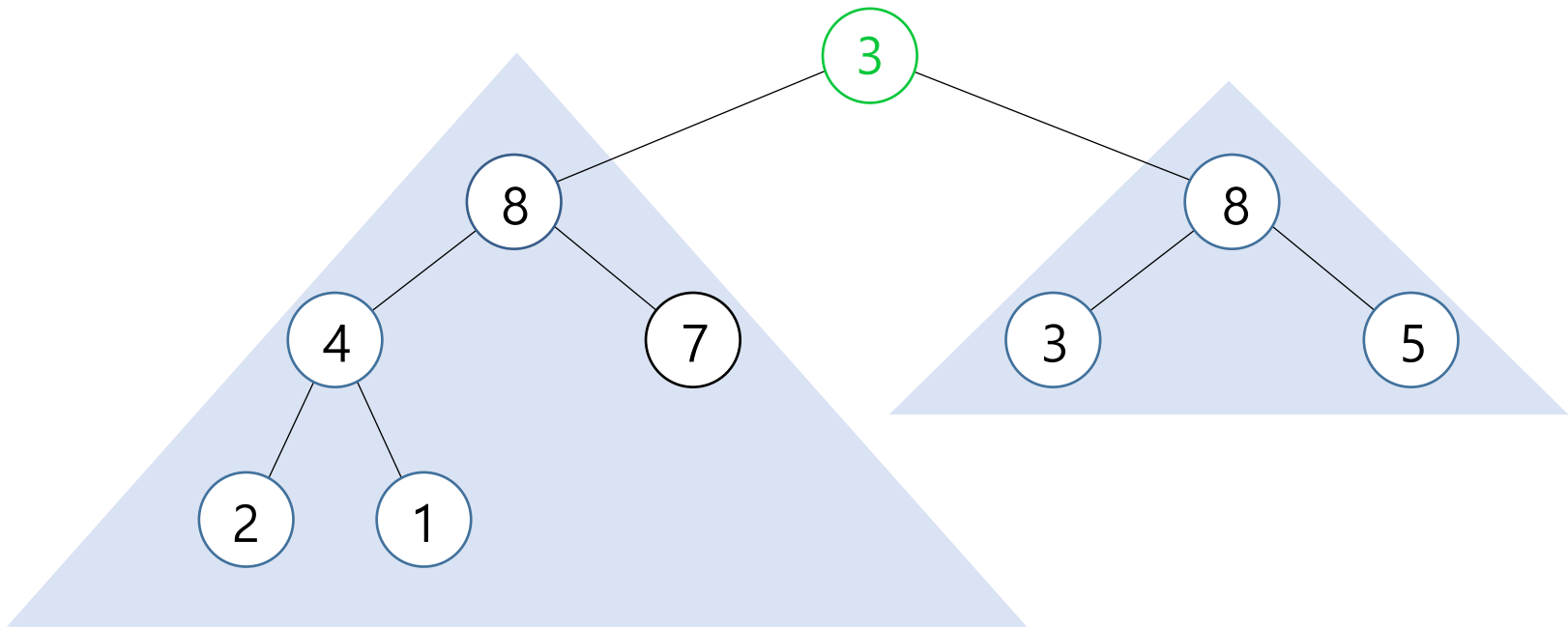
- 원소 삭제



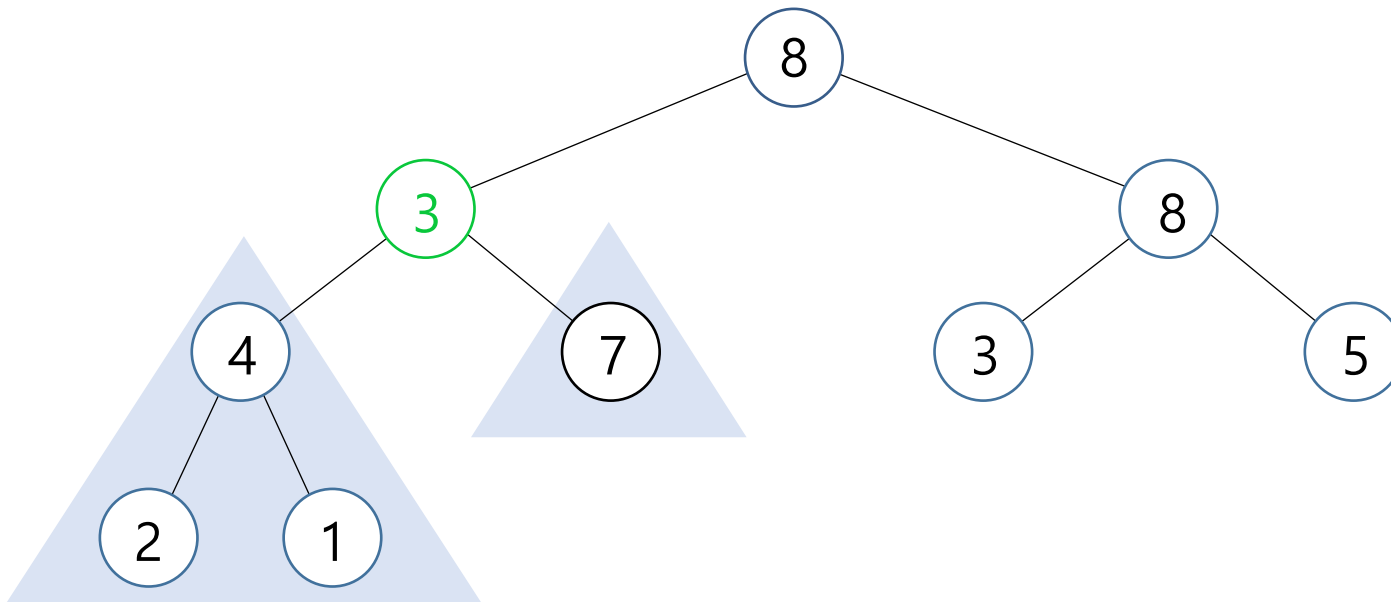
- 힙의 작업과 구현
 - 원소 삭제



- 힙의 작업과 구현
 - 원소 삭제



- 힙의 작업과 구현
 - 원소 삭제



- 힙의 작업과 구현

- 원소 삭제

```
deleteMax():  
    max ← A[0]  
    A[0] ← A[n-1]  
    n--  
    percolateDown(0)  
    return max
```

```
percolateDown(k):  
    child ← 2k+1  
    right ← 2k+2  
    if (child ≤ n-1)  
        if (right ≤ n-1 and A[child] < A[right])  
            child ← right  
    if (A[k] < A[child])  
        A[k] ↔ A[child]  
        percolateDown(child)
```

- 힙의 작업과 구현

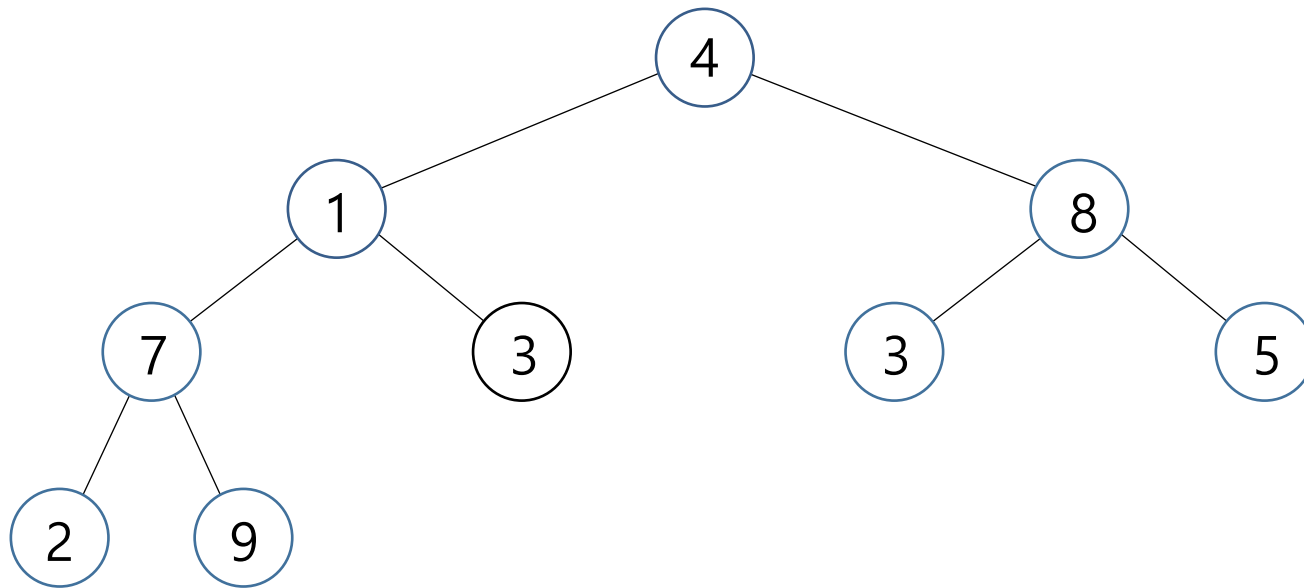
- 원소 삭제

```
def deleteMax(self):
    if (not self.isEmpty()):
        max = self.__A[0]
        self.__A[0] = self.__A.pop()
        self.percolateDown(0)
        return max

def percolateDown(self, i:int):
    child = 2*i + 1
    right = 2*i + 2
    if child <= len(self.__A)-1:
        if right <= len(self.__A)-1 and A[child] < A[right]:
            child = right
        if self.__A[i] < self.__A[child]:
            self.__A[i], self.__A[child] = self.__A[child], self.__A[i]
            self.__percolateDown(child)
```

- 힙의 작업과 구현

- 힙 생성

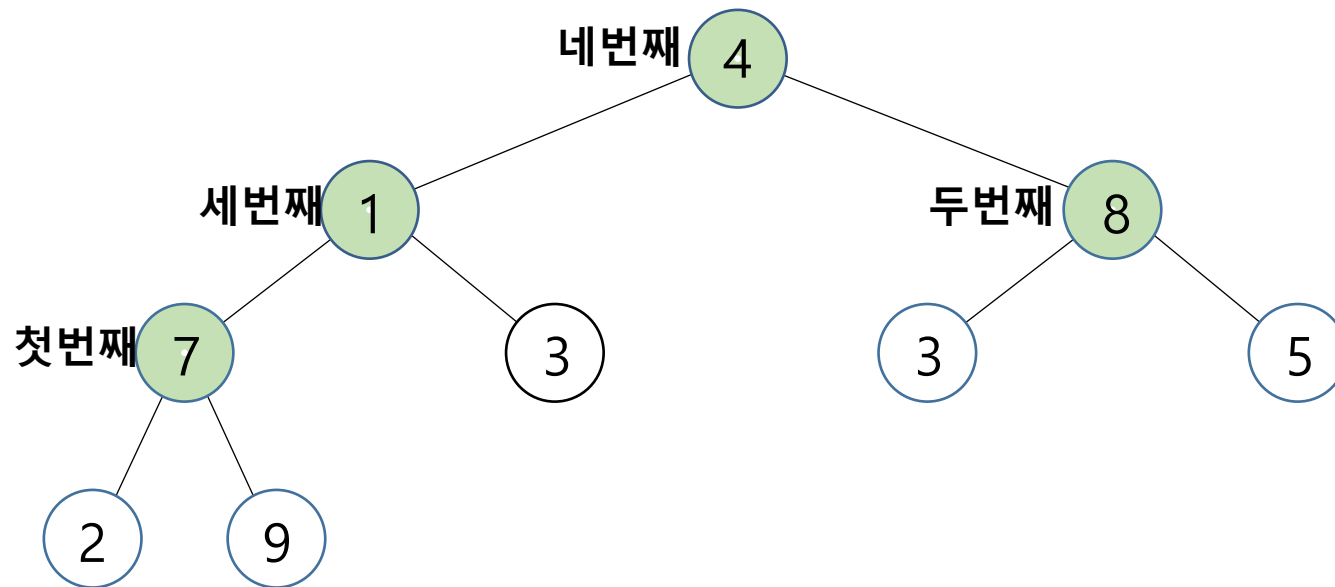


4	1	8	7	3	3	5	2	9
---	---	---	---	---	---	---	---	---

■ 힙의 작업과 구현

- 힙 생성

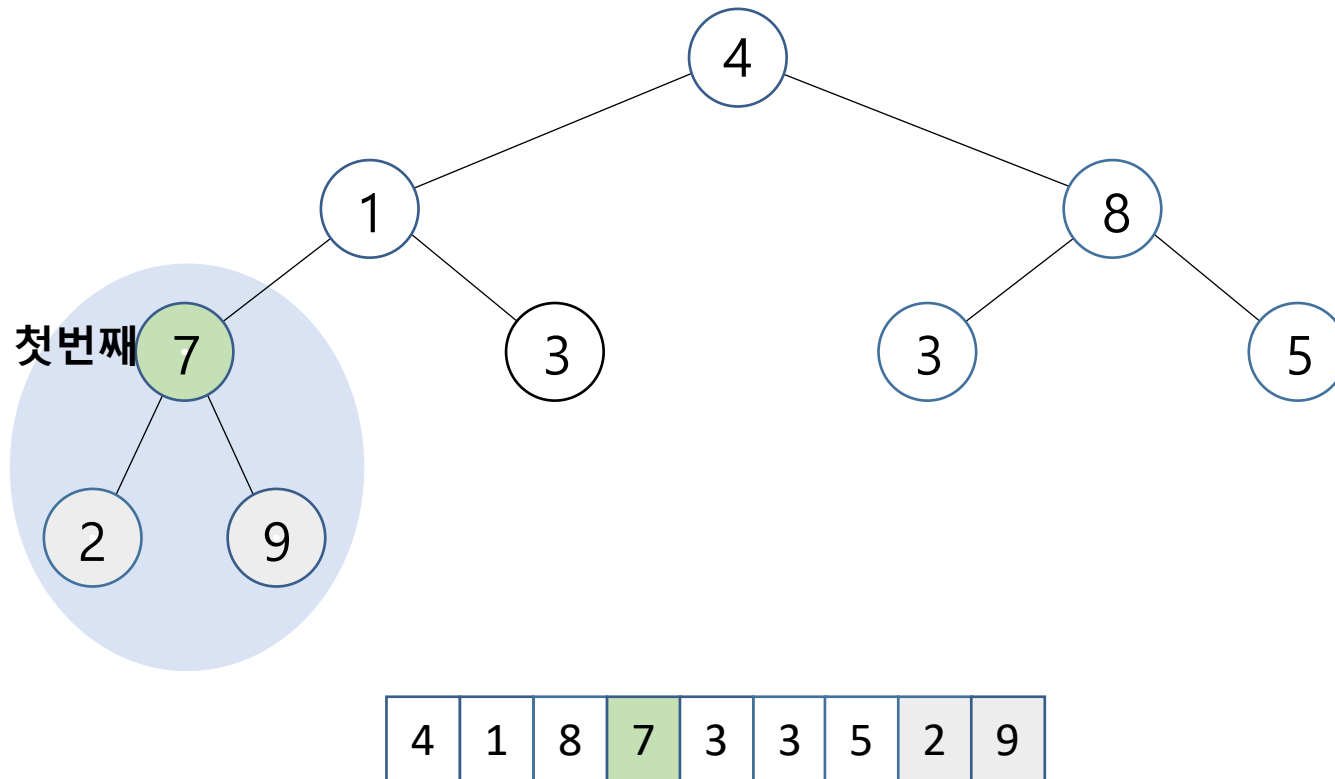
부모 노드를 루트로 하는 서브 트리를 힙으로 수



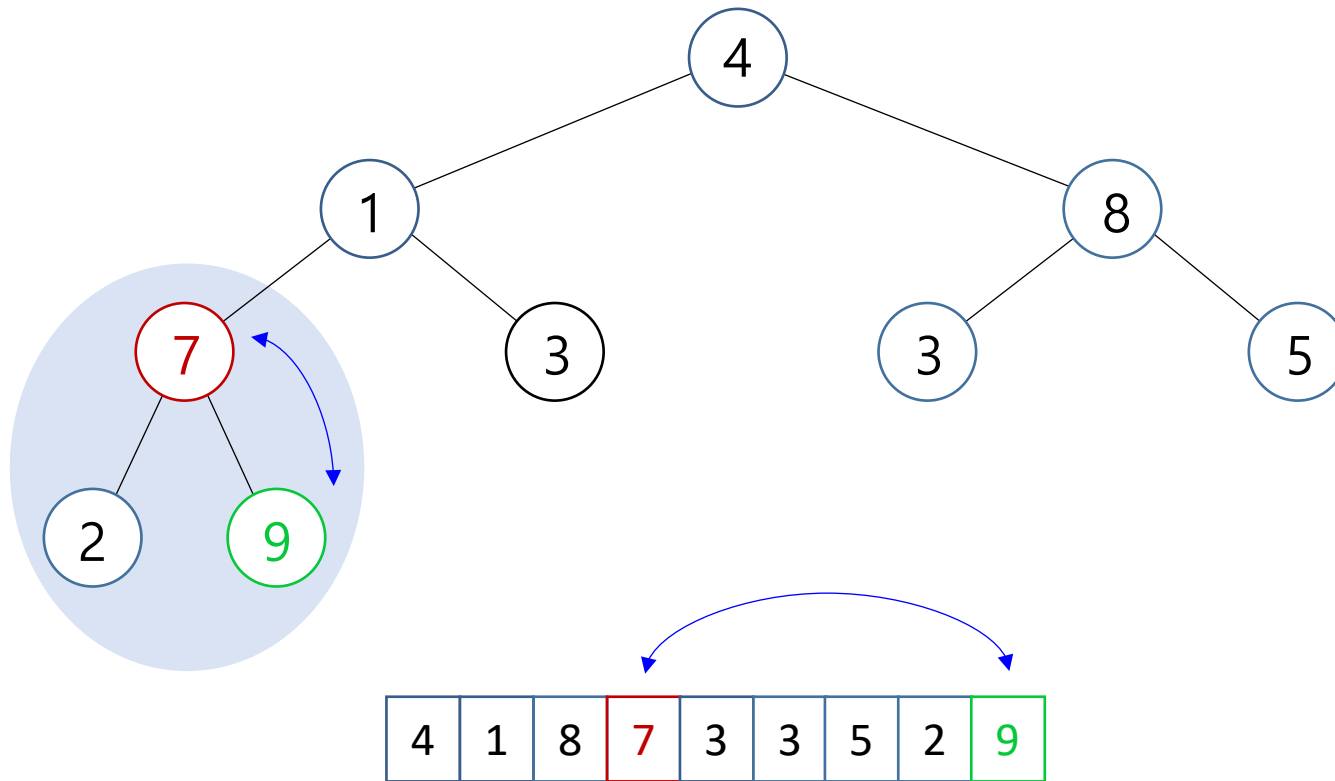
4	1	8	7	3	3	5	2	9
---	---	---	---	---	---	---	---	---

- 힙의 작업과 구현

- 힙 생성

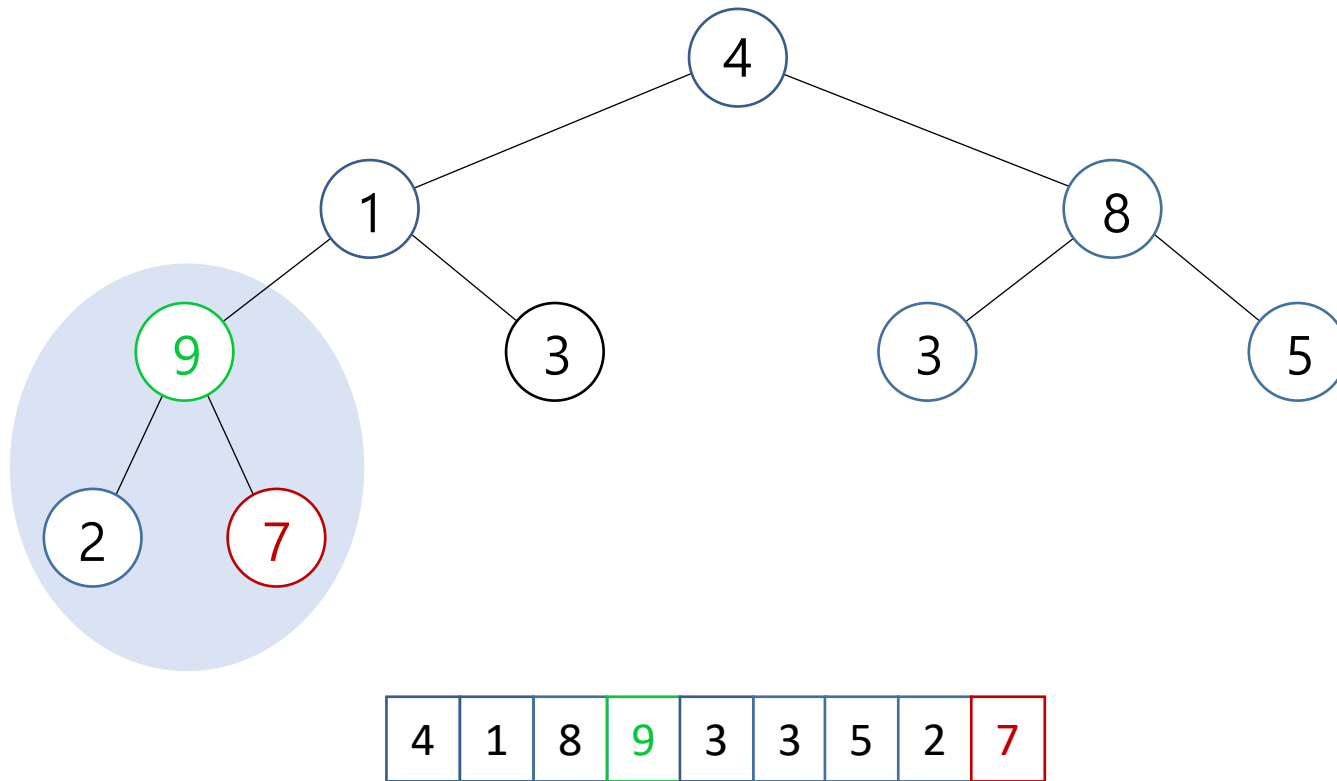


- 힙의 작업과 구현
 - 힙 생성



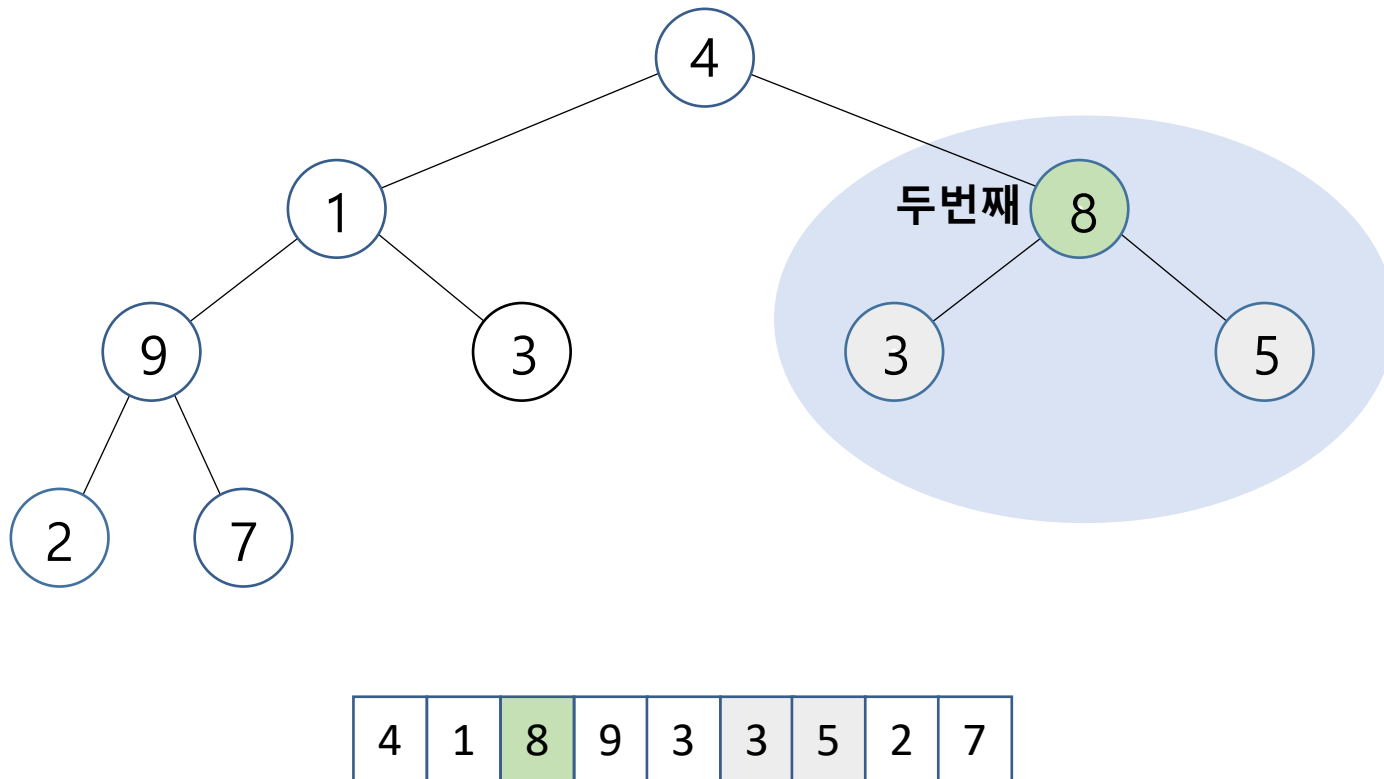
- 힙의 작업과 구현

- 힙 생성

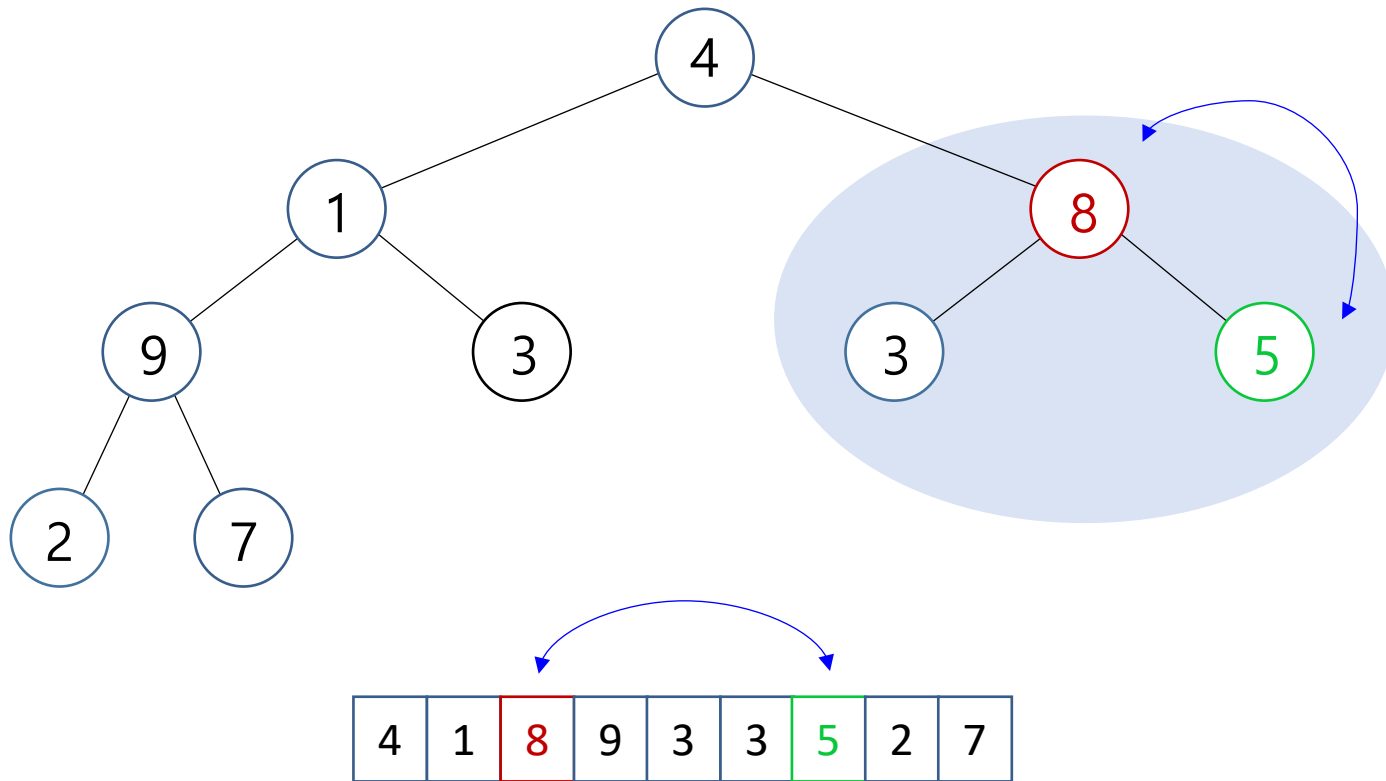


- 힙의 작업과 구현

- 힙 생성

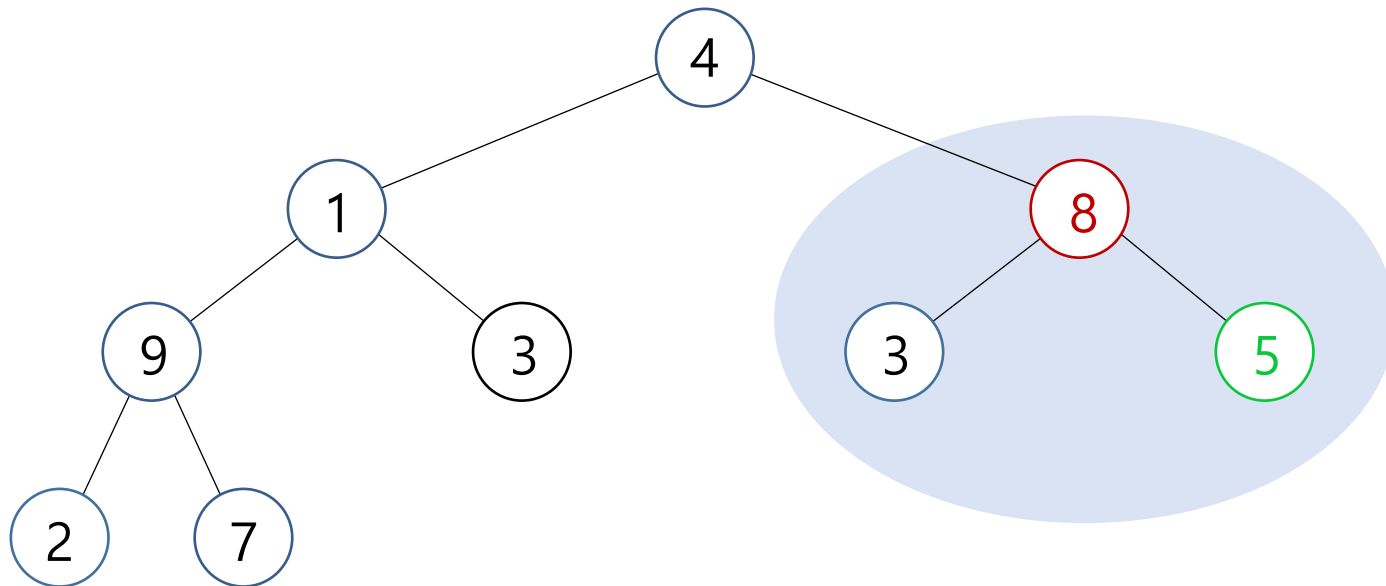


- 힙의 작업과 구현
 - 힙 생성



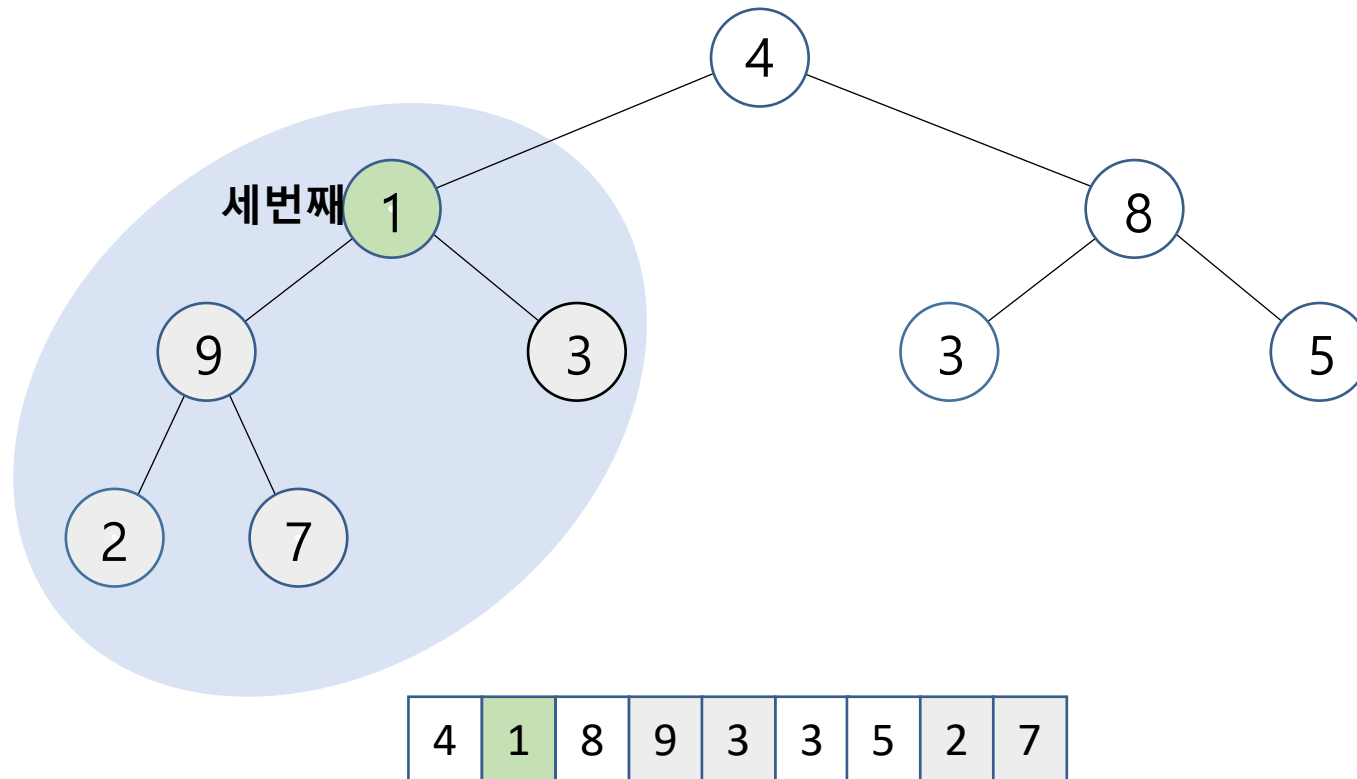
- 힙의 작업과 구현

- 힙 생성



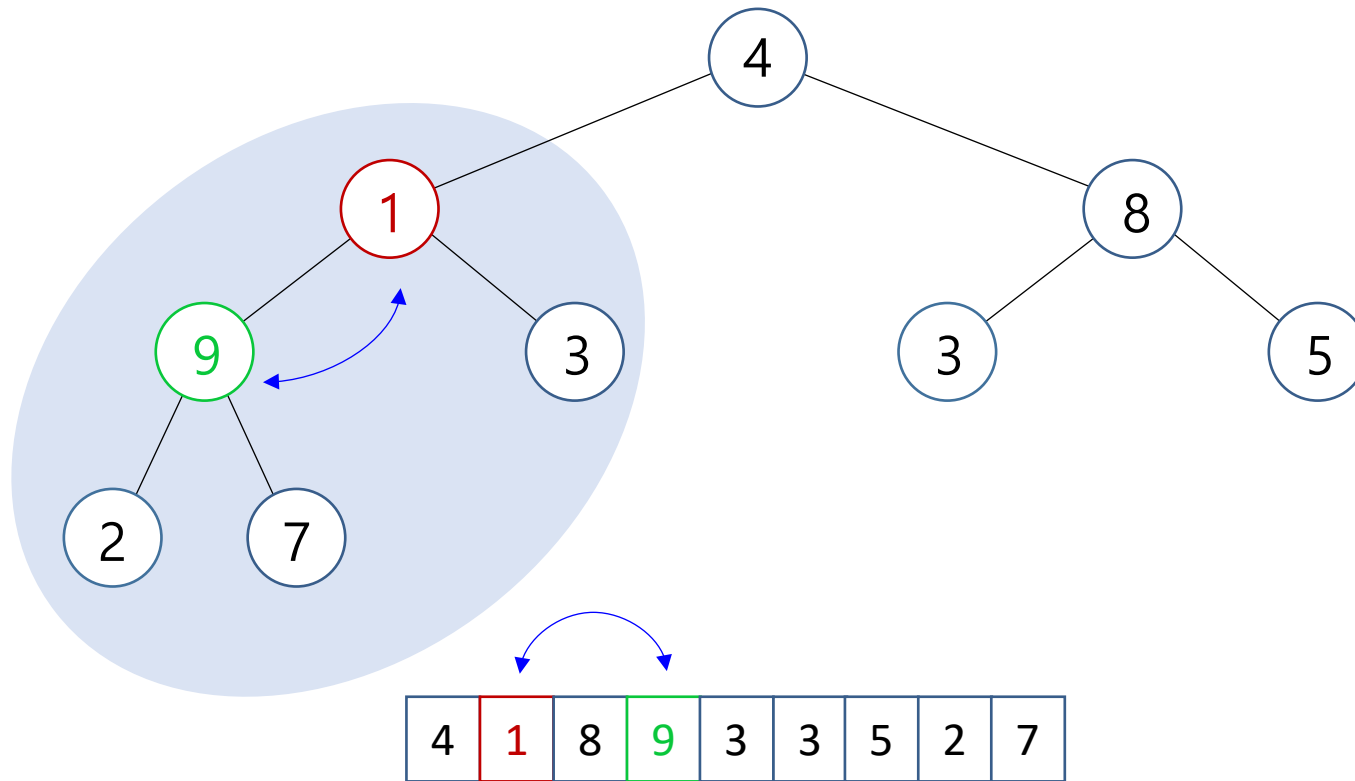
- 힙의 작업과 구현

- 힙 생성



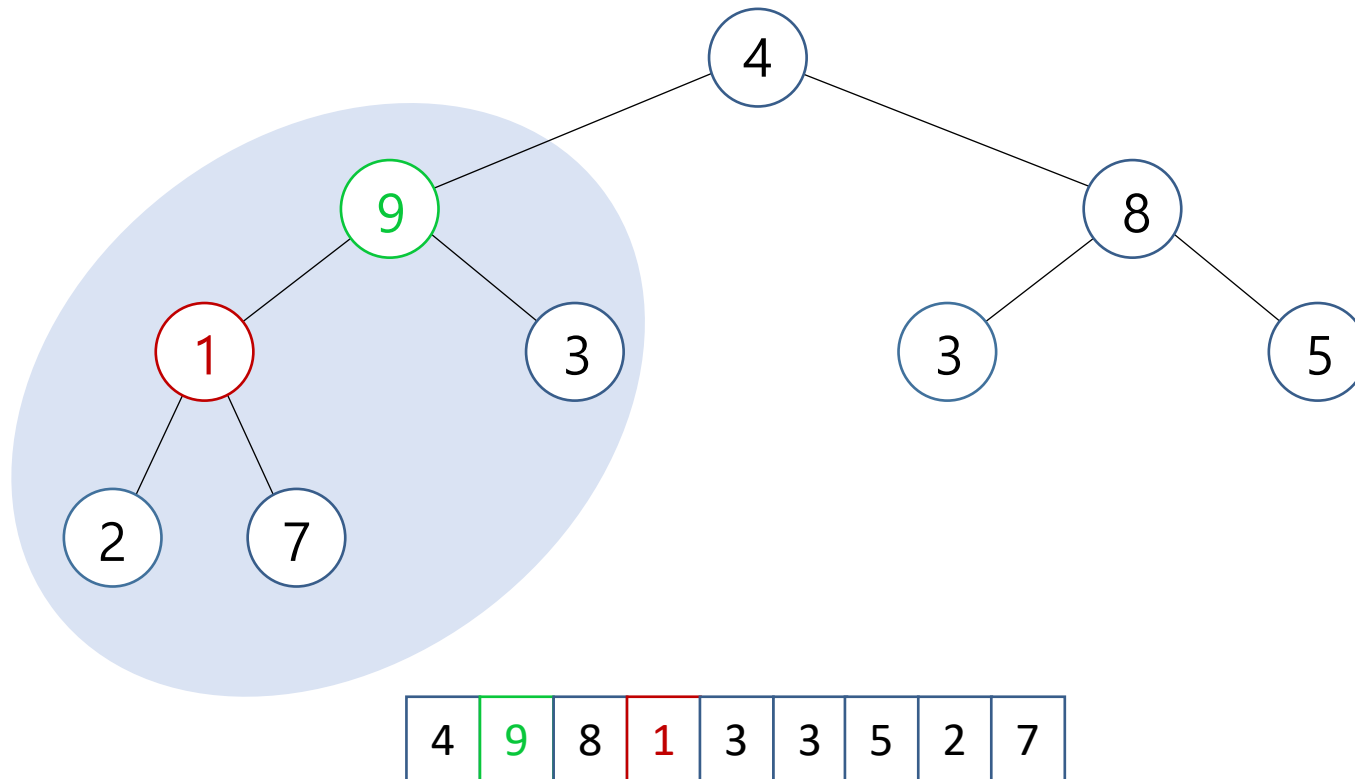
- 힙의 작업과 구현

- 힙 생성



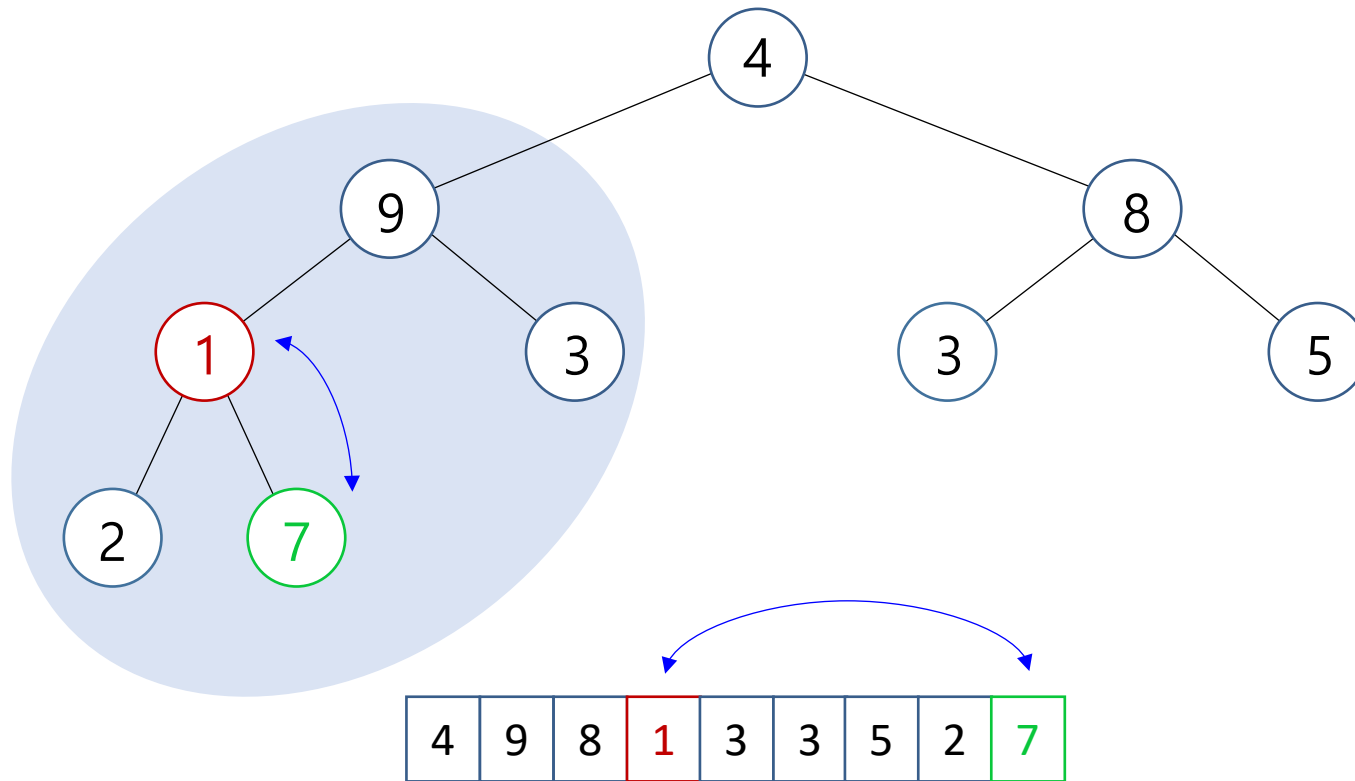
- 힙의 작업과 구현

- 힙 생성



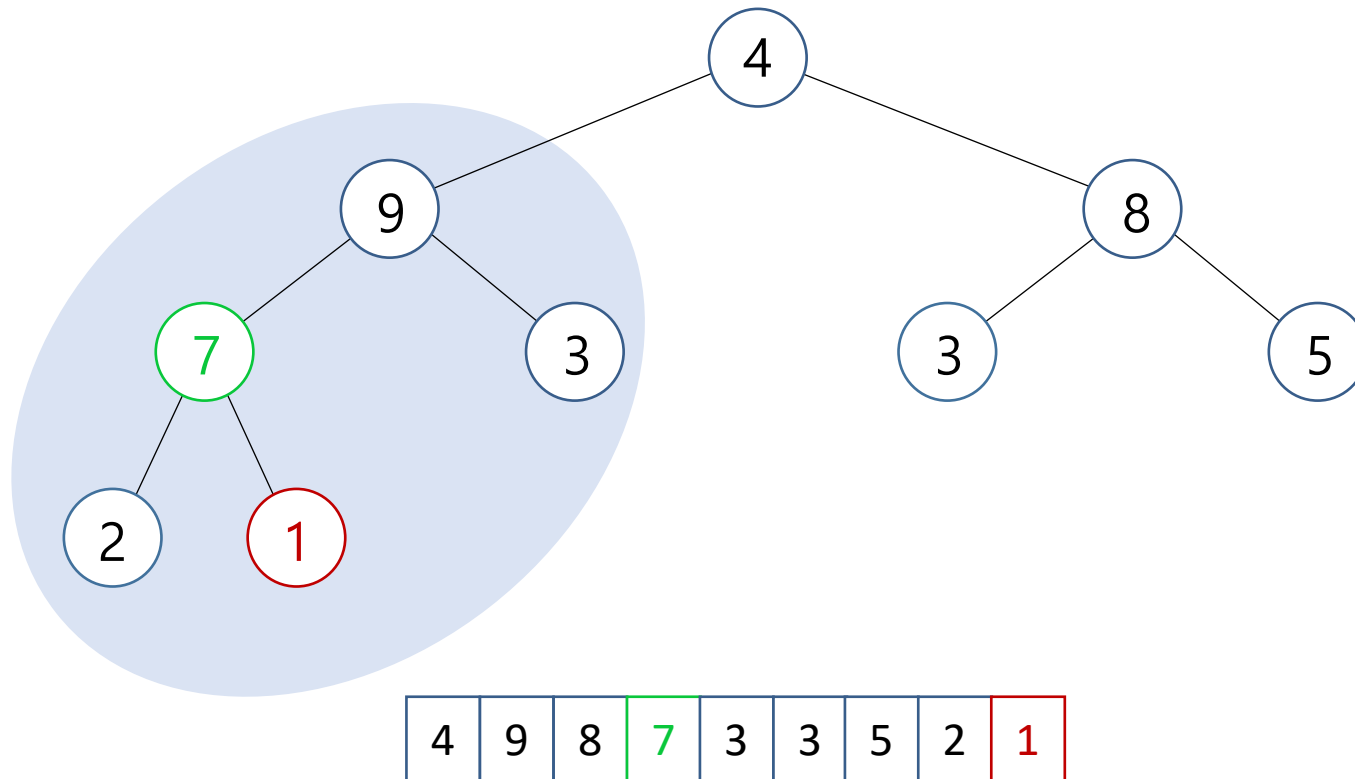
- 힙의 작업과 구현

- 힙 생성



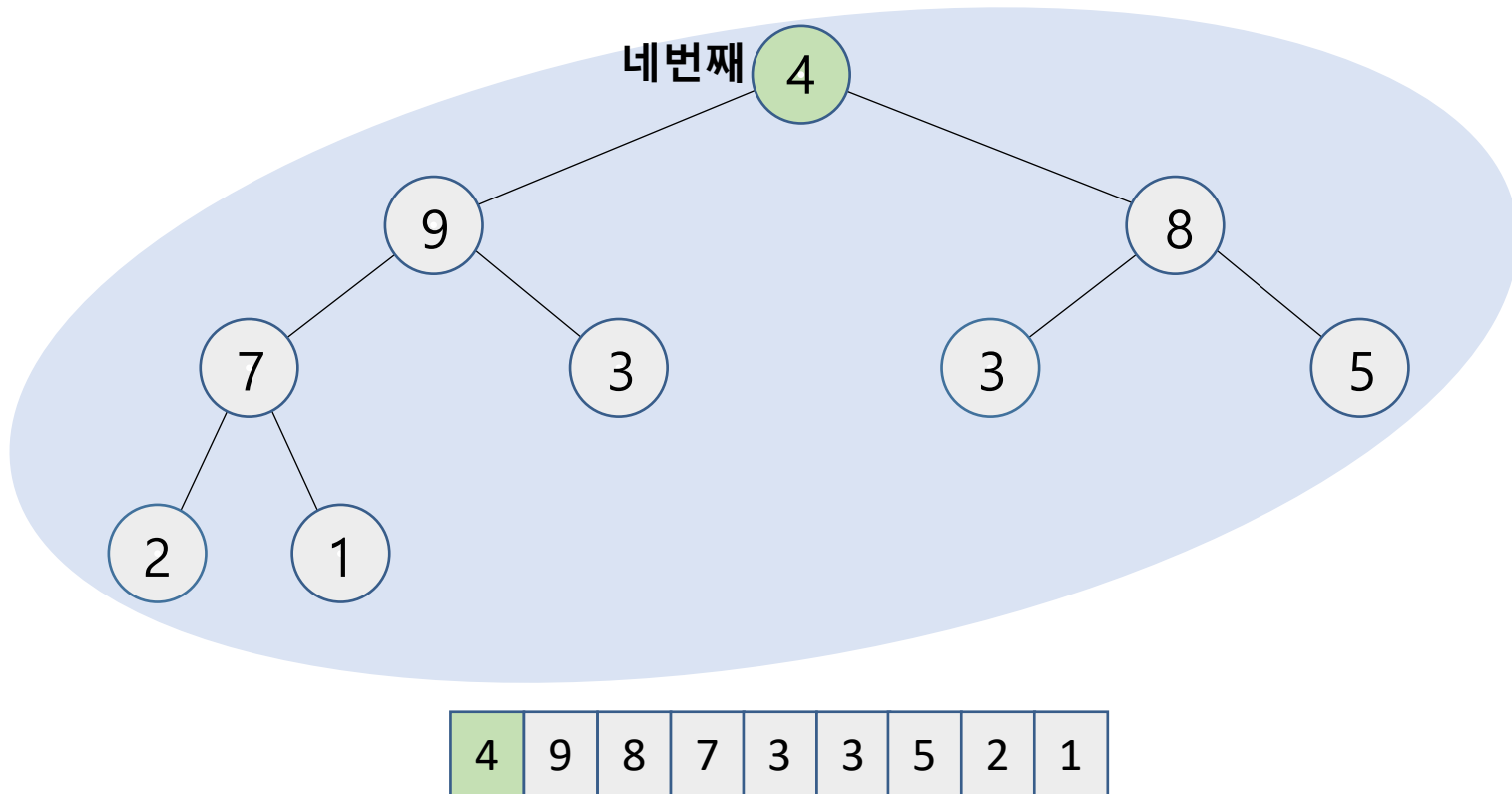
- 힙의 작업과 구현

- 힙 생성



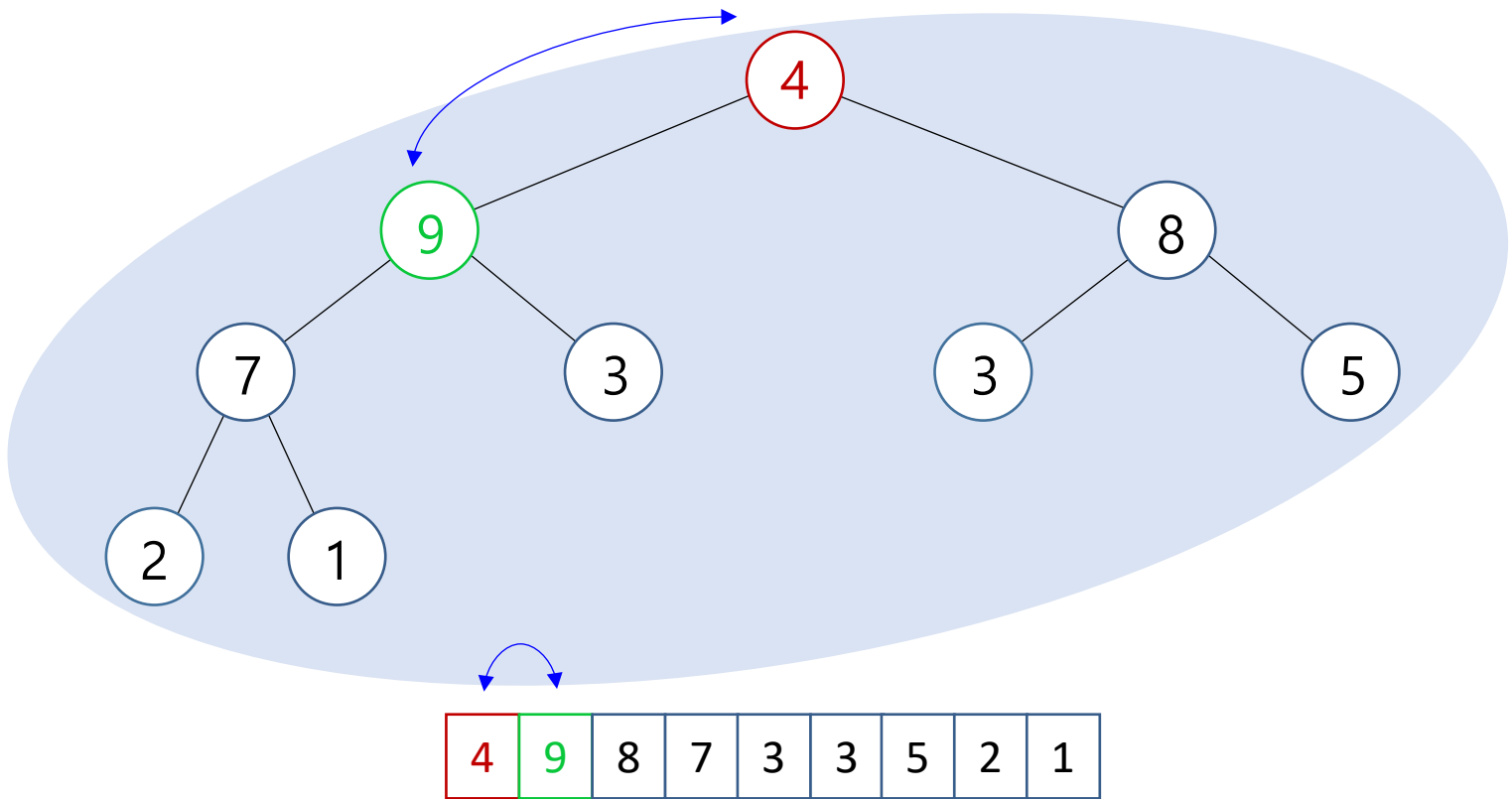
- 힙의 작업과 구현

- 힙 생성



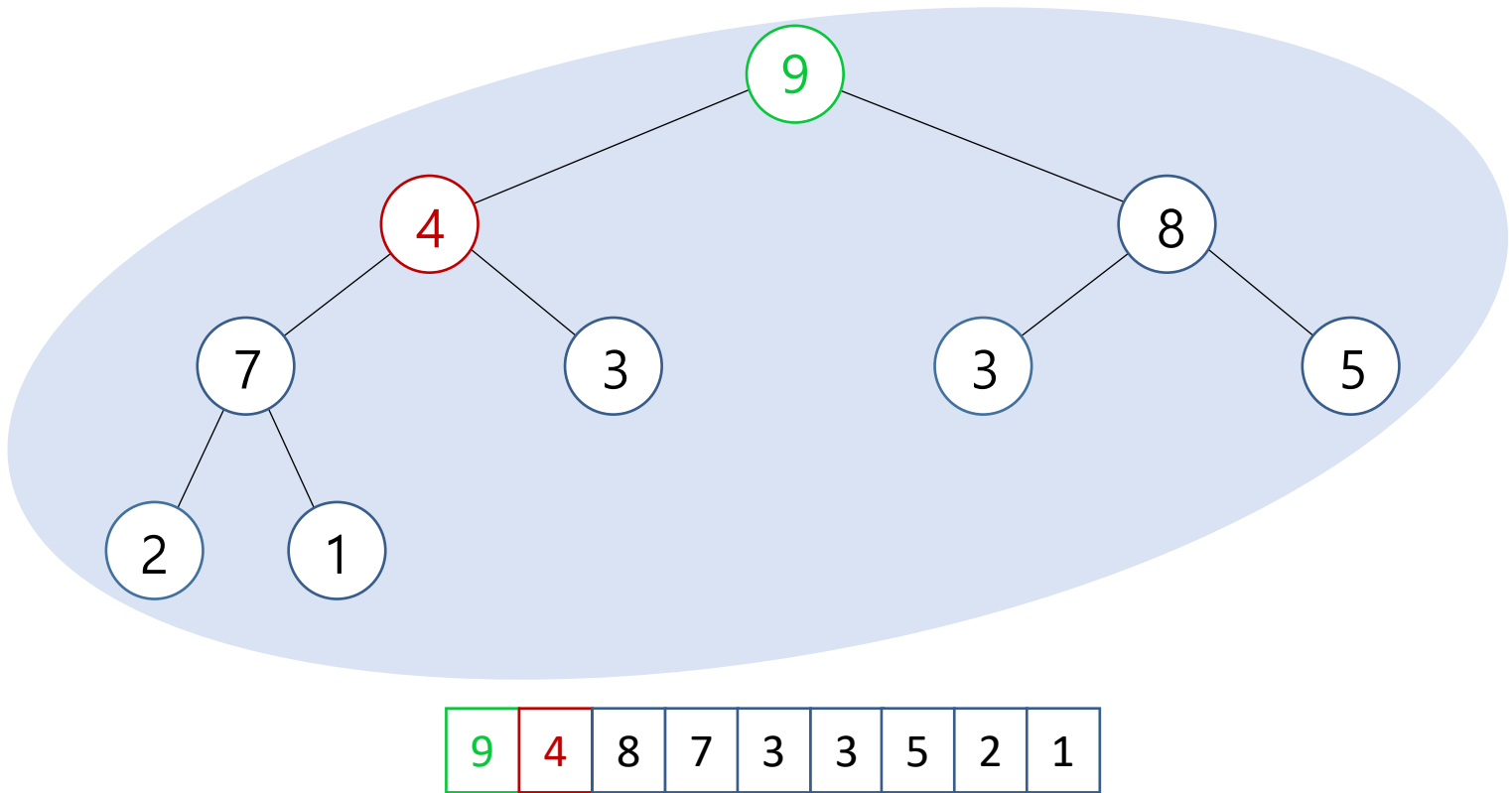
- 힙의 작업과 구현

- 힙 생성



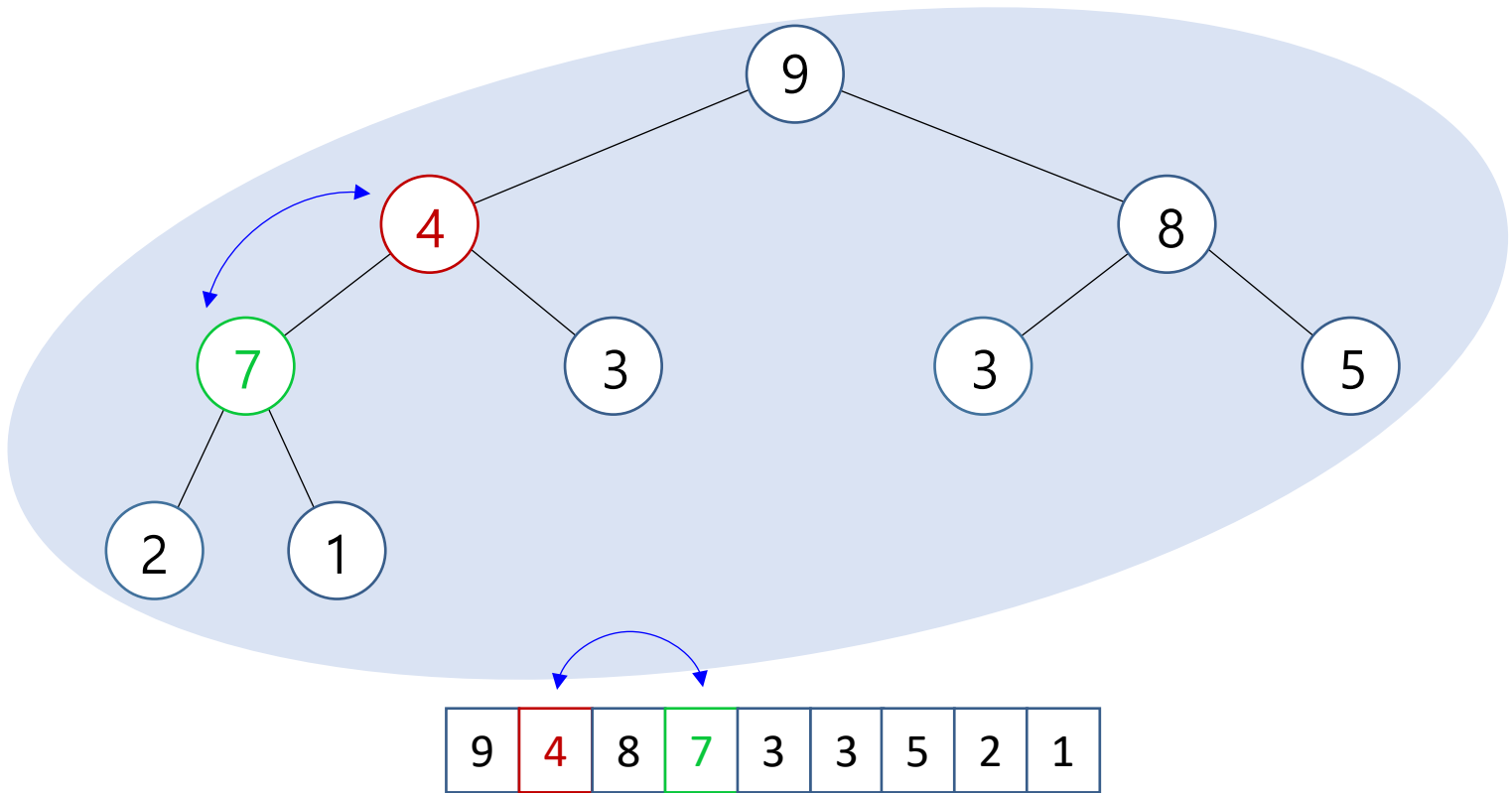
- 힙의 작업과 구현

- 힙 생성



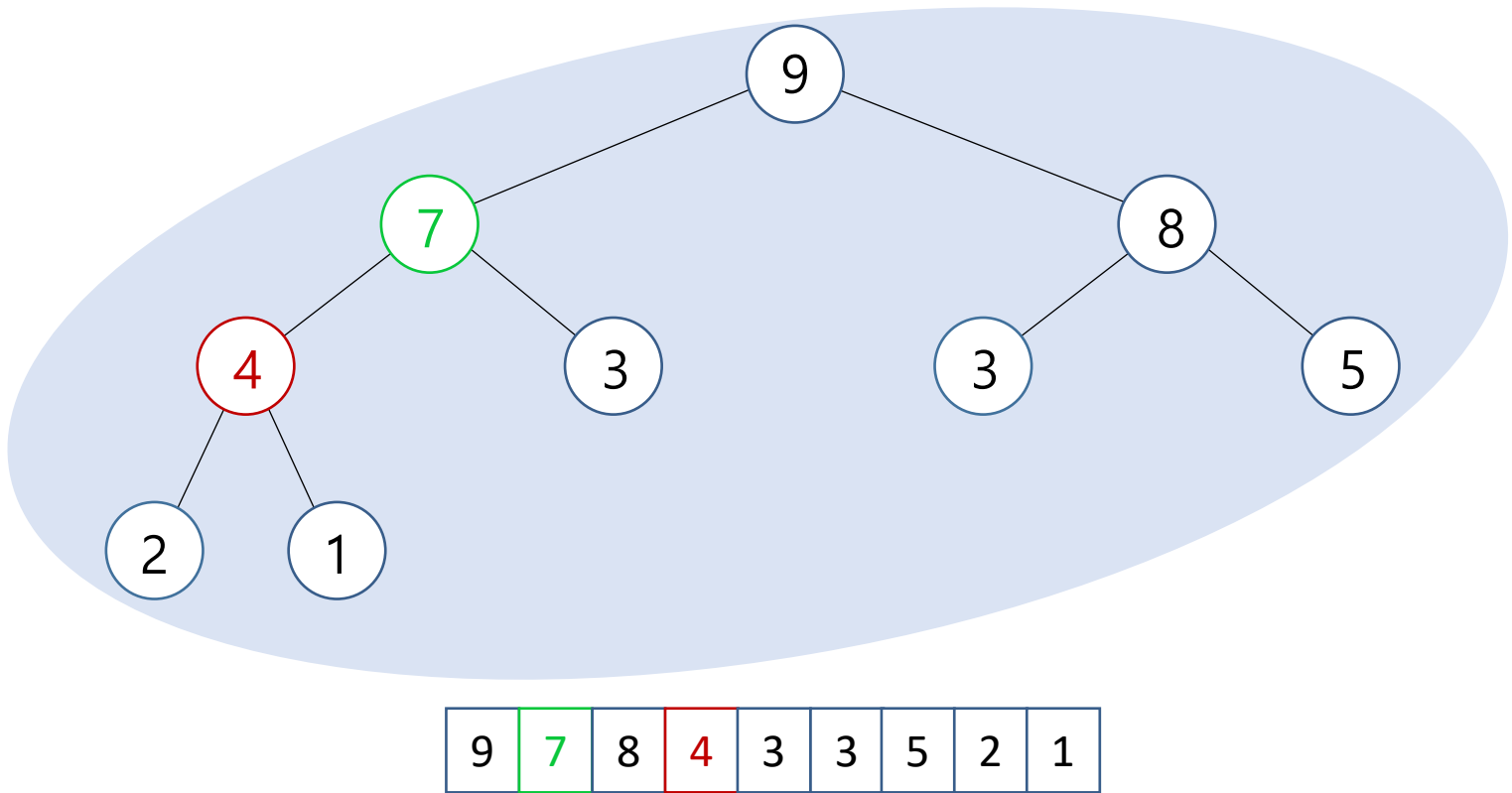
- 힙의 작업과 구현

- 힙 생성



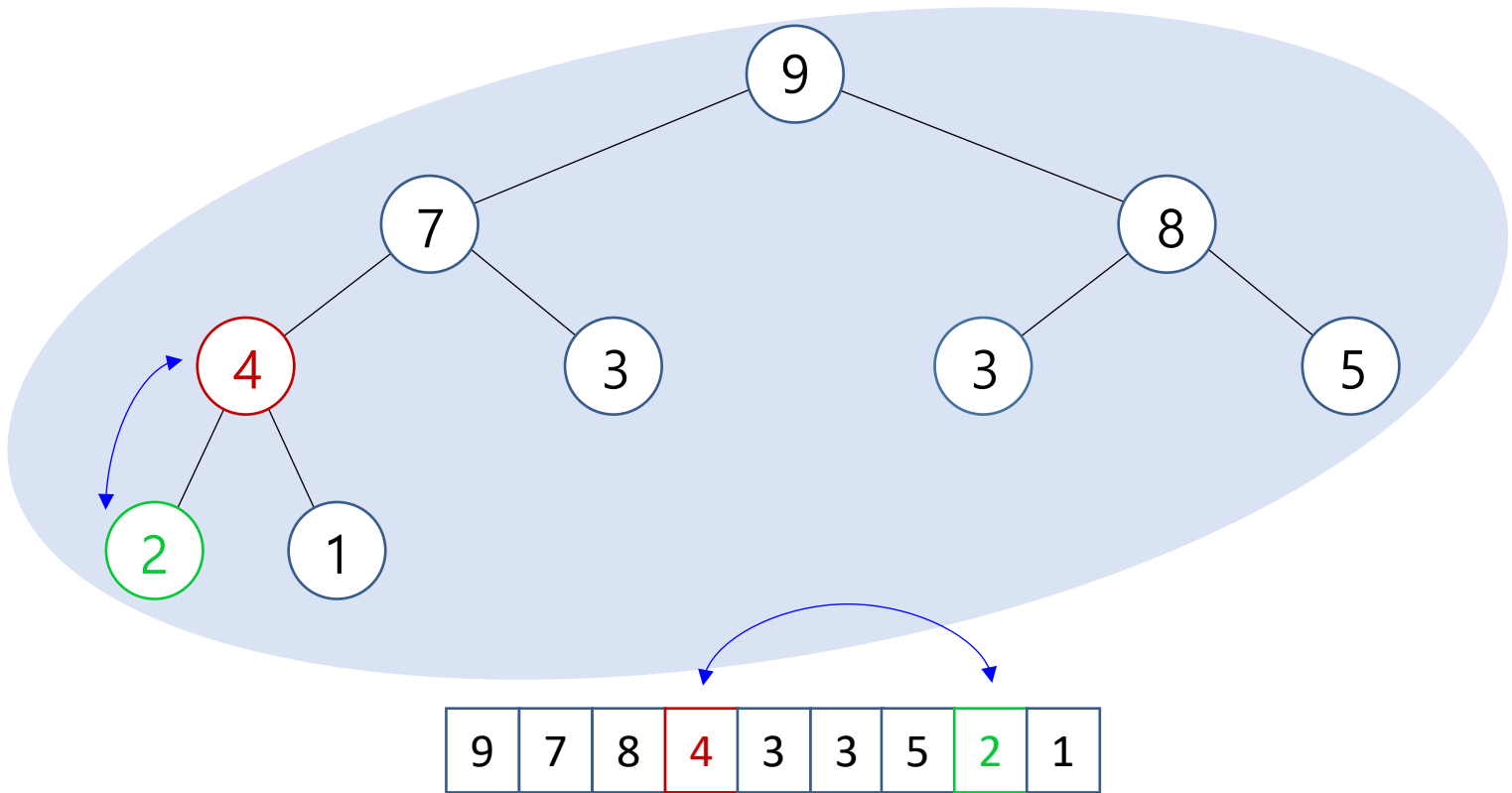
- 힙의 작업과 구현

- 힙 생성



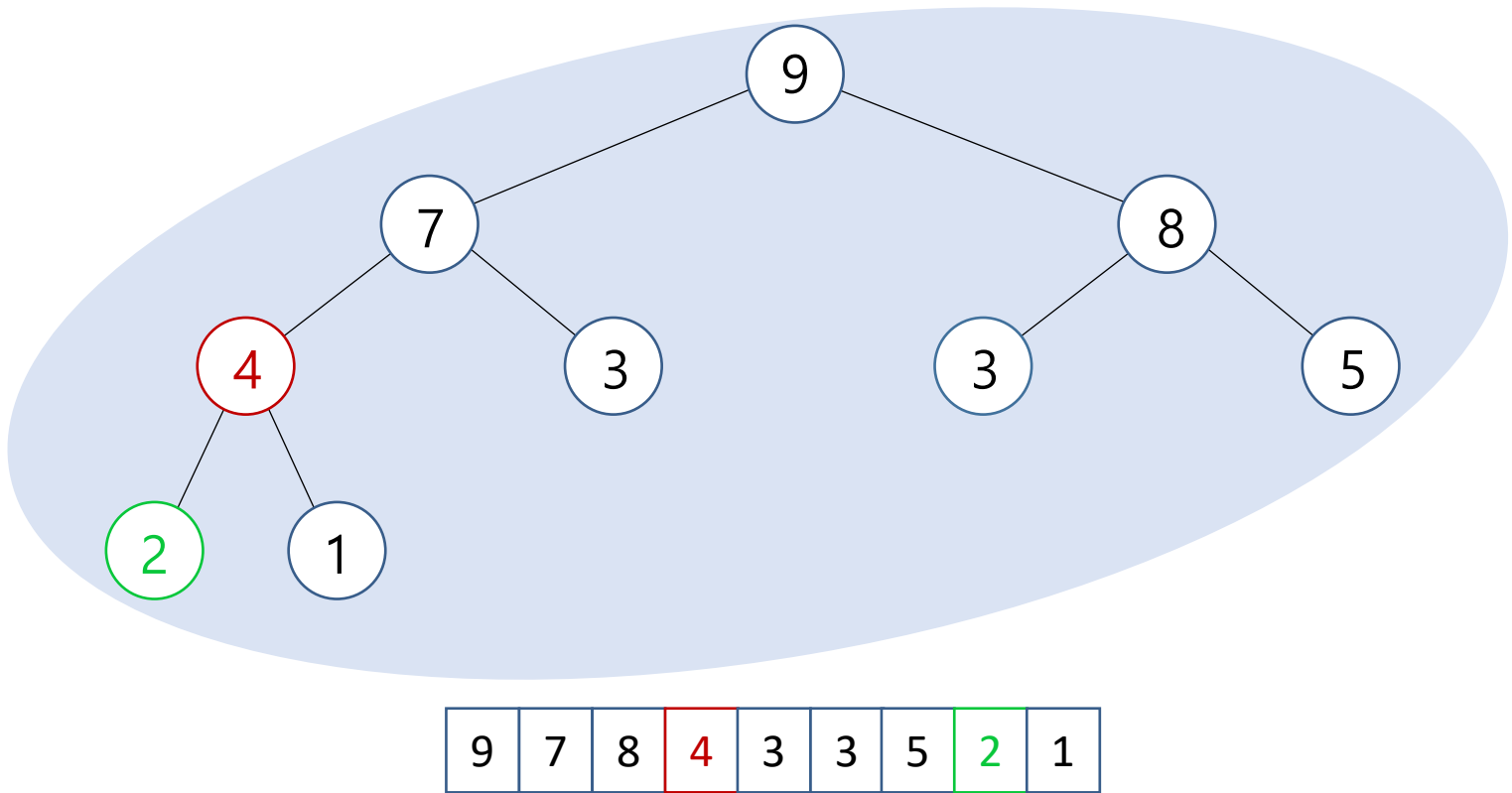
- 힙의 작업과 구현

- 힙 생성



- 힙의 작업과 구현

- 힙 생성



- **힙의 작업과 구현**
 - 힙 생성

```
def buildHeap(self):  
    for i in range((len(self.__A) - 2) // 2, -1, -1):  
        self.percolateDown(i)
```

- **힙의 작업과 구현**
 - 기타 작업

```
def max(self):  
    return self.__A[0]
```

```
def isEmpty(self):  
    return len(self.__A)
```

```
def clear(self):  
    self.__A = []
```

3. 힙 수행 시간

- 힙 수행 시간

- buildHeap

- percolateDown 의 시간을 모두 합친 것 : $\Theta(n)$

- percolateDown 은 총 $\lfloor n/2 \rfloor$

- $n/2$ 개는 1레벨

- $n/4$ 개는 2레벨

- $n/8$ 개는 3레벨 ...

- 1 개는 $\lfloor \log_2 n \rfloor$ 레벨

- Insert

- 한 번의 percolateUp : $O(\log n)$

- deleteMax

- 한 번의 percolateDown : $O(\log n)$